Module 2

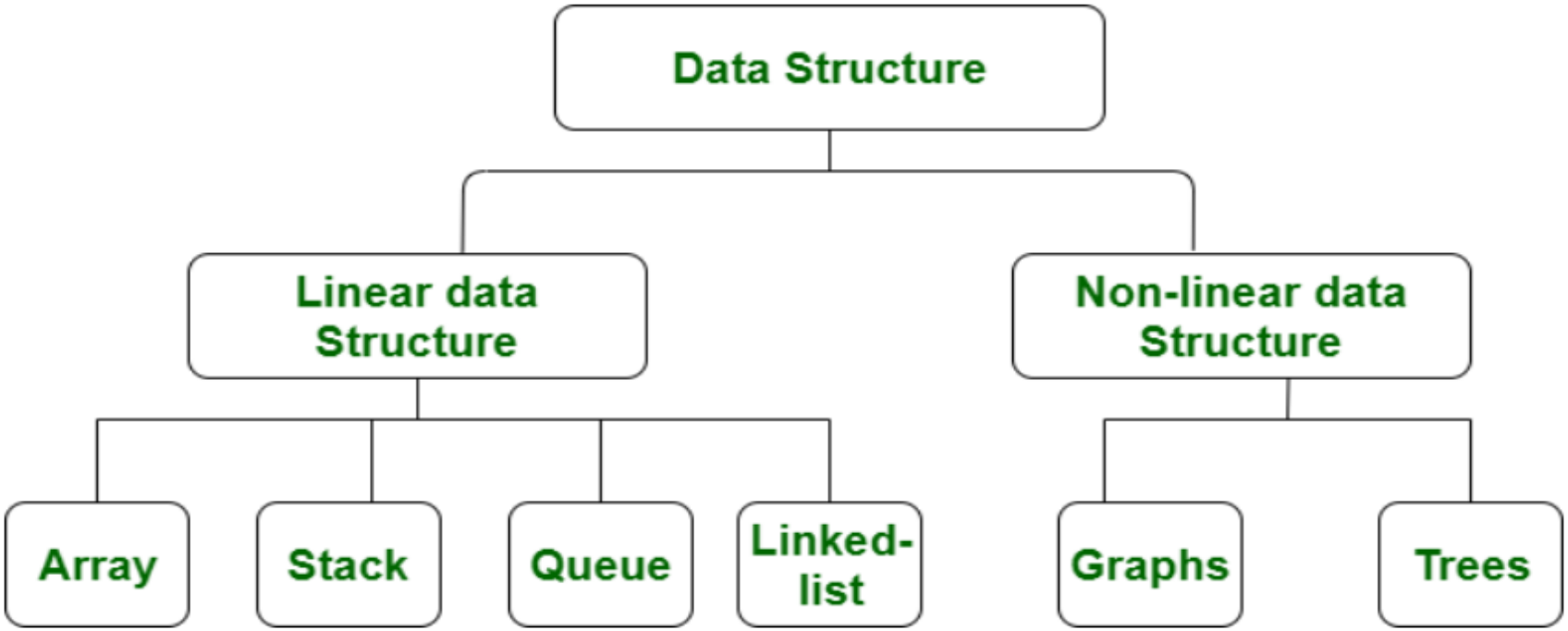
LINEAR DATA STRUCTURES

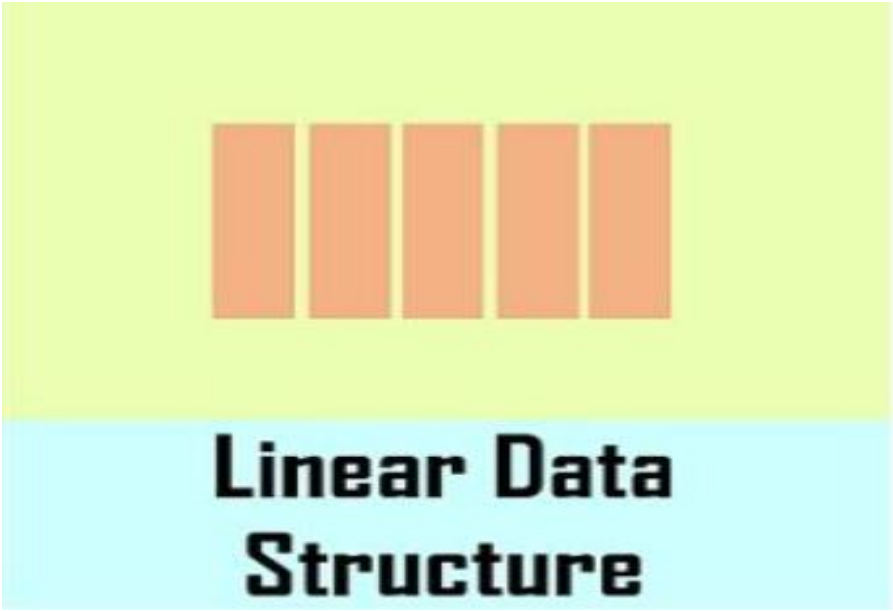
Data structures

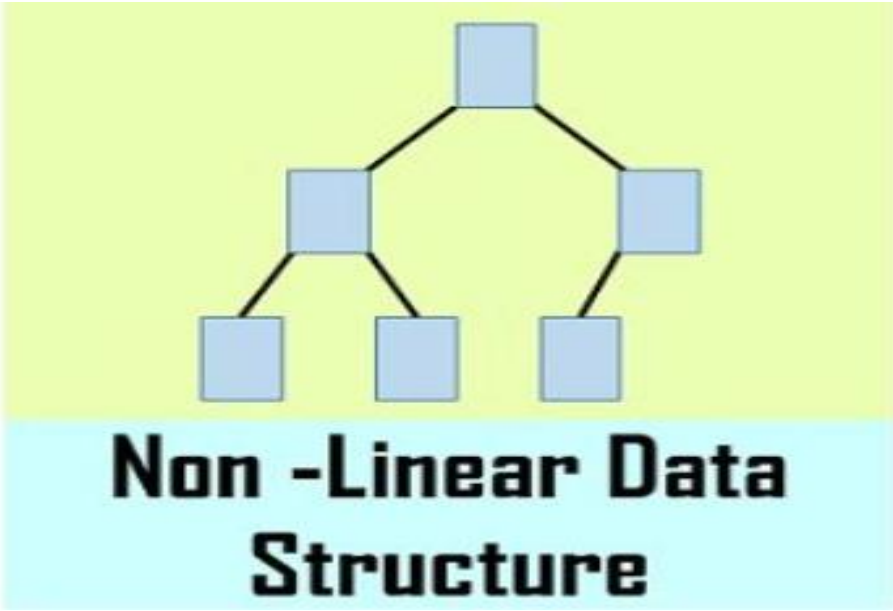
Data structure is a way of organizing the data in the computer system so that it can be used efficiently.

Types of Data structures

1. **Linear** - store data in a sequential manner
2. **Non Linear** - organize data hierarchically


Data Structure
Linear data Structure
Non-linear data Structure
Array
Stack
Queue
Linked-list
Graphs
Trees


Linear Data Structure


Non -Linear Data Structure

Linear data structure:

A data structure is said to be linear if its elements form a sequence or a linear list.

Eg. **Array, linked list, stack, queue.**

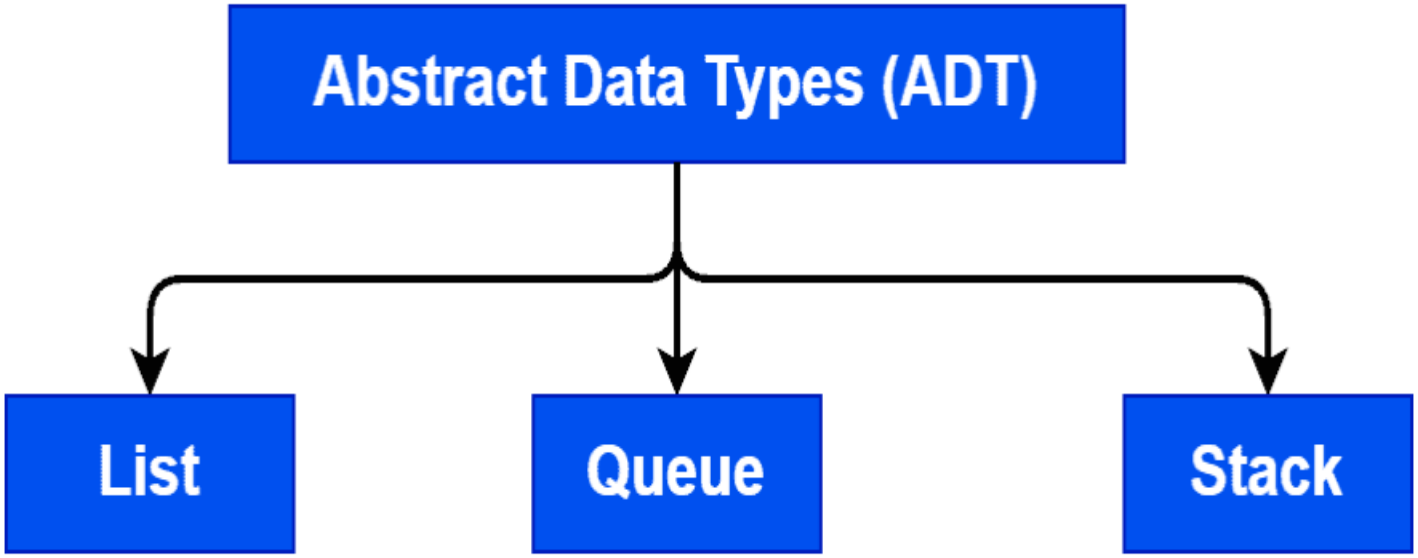
Non Linear data structure

A data structure is said to be non linear if its elements are not in a sequence.

Eg. **Tree, graph**

Abstract data type (ADT):

An **Abstract Data Type (ADT)** is a conceptual model that defines a set of operations and behaviors for a data structure.



Array Disadvantages

1. **Fixed Size**
2. **Sequential Access**
3. **Insertion and Deletion is costly**

List ADT - Linked List

Linked List is a Linear data structure where elements, called nodes, are linked together using pointers are references.

Each node contains the following **two fields**:

1. **Data Field**
2. **Pointer or Address Field**

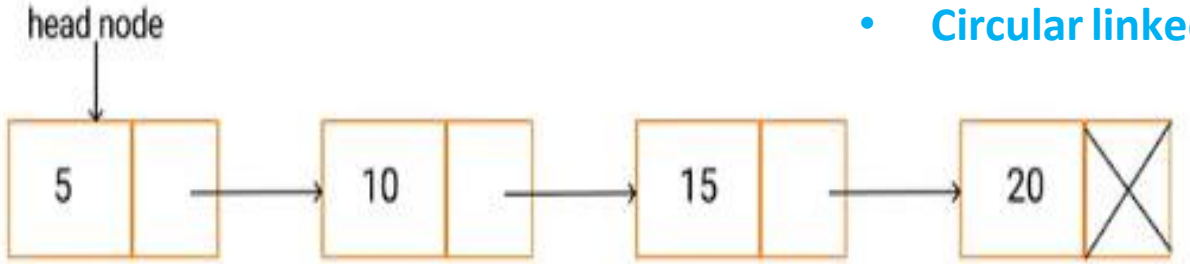
- **Data field contains the actual information** that has to be stored
- **Address field contains the address of the next node**.
- The size of the list get vary when an element is inserted or deleted

Types of Linked List

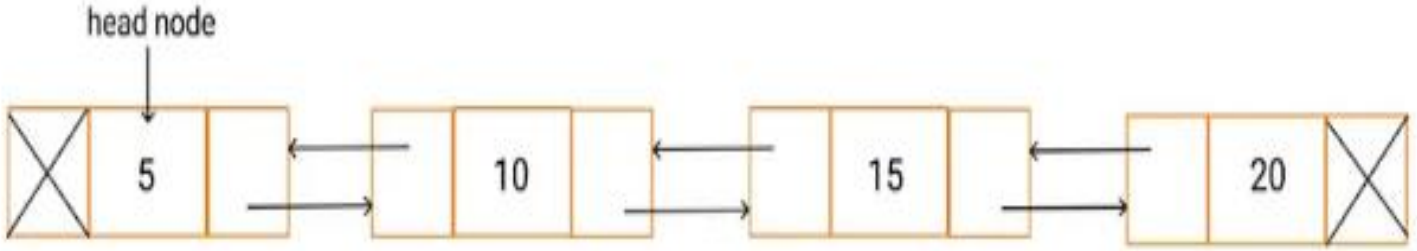
Types of linked list:

- **Singly linked list**
- **Doubly linked list**
- **Circular linked list**

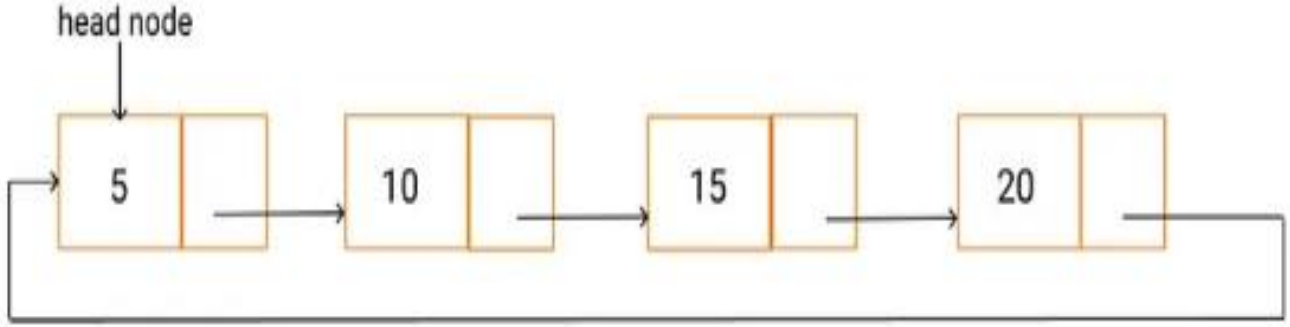
Singly Linked List



Doubly Linked List



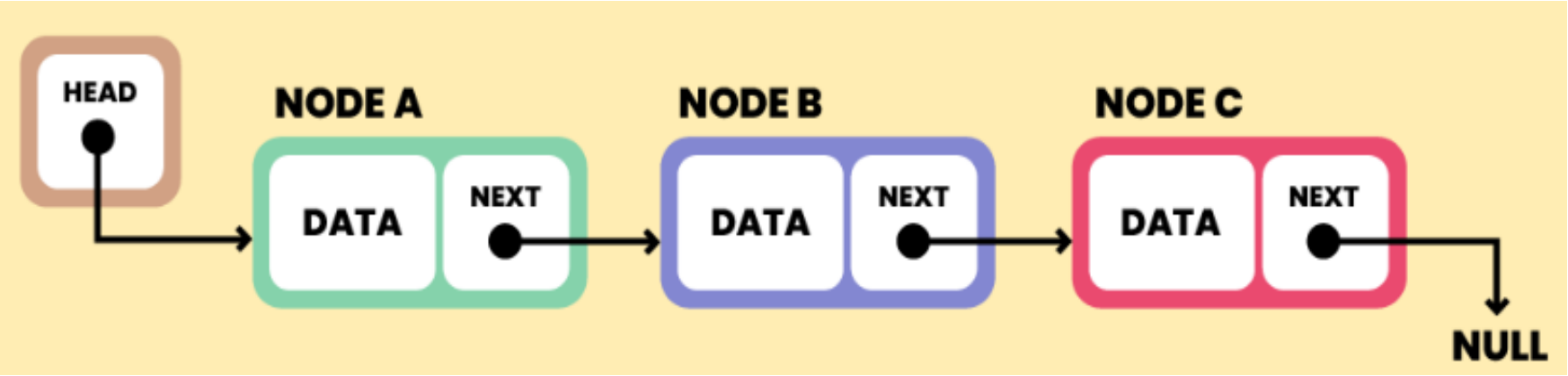
Circular Linked List

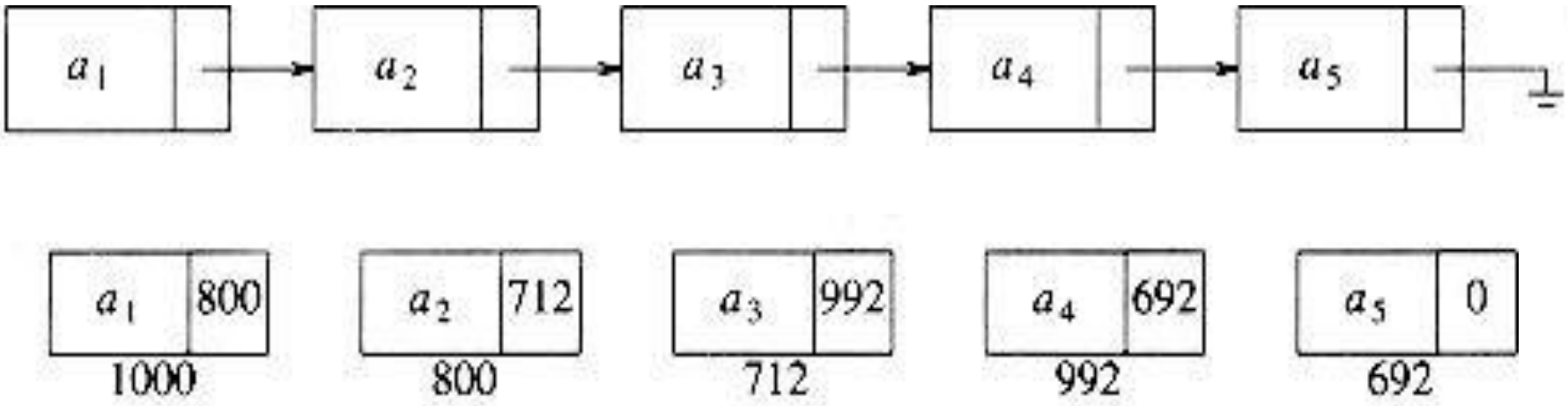


List ADT

- A List is the dynamic collection of items or elements A1,A2,A3…An of size n arranged in a linear sequence.
- Ai refers to the ith element in the list.
- A list with size zero or with no elements is referred as **empty list.**
- For any list Ai-1 precedes Ai where i>1 and Ai+1 follows Ai where i<n.

Address field of Last node contains the null value to indicate the end of the list





Representation of a node in linked list

```
struct node
{
        int data;
        struct node *next;
};
struct node *head;
```

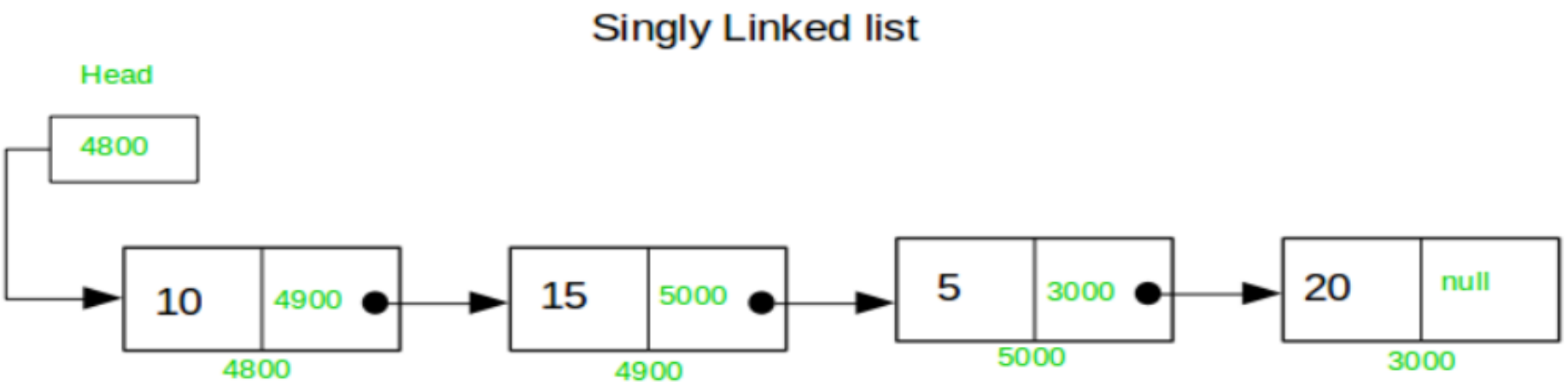
Advantage of linked list:

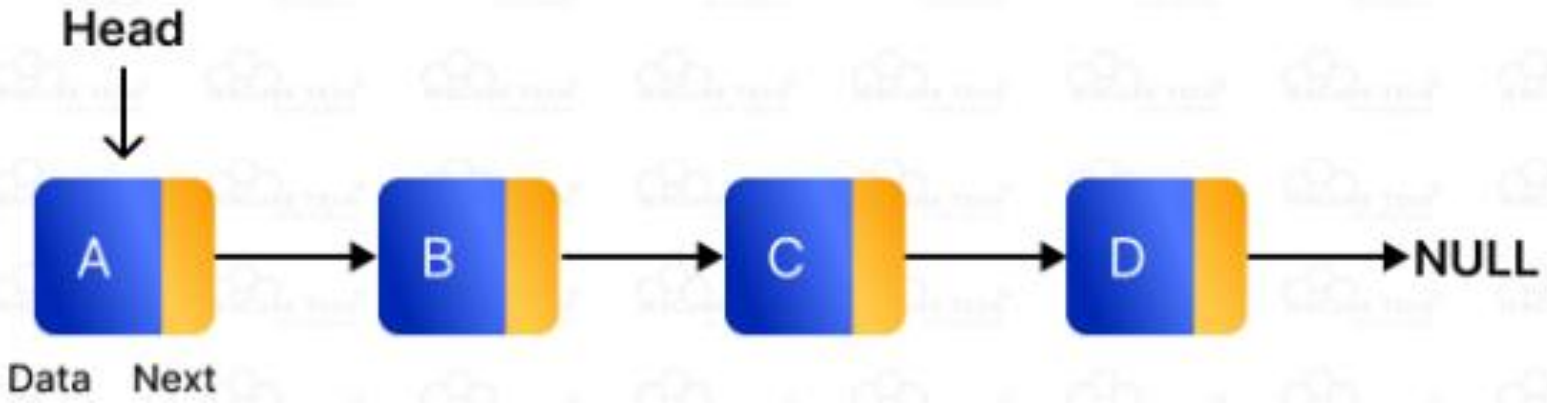
- Insertion and deletion can be done in efficient manner.
- Memory is allocated during the runtime.

Singly linked list

- It is a linear data structure.
- Singly linked list contains collection of nodes where each and every node contain data and address field.
- Data field contain data information in any type.
- Address field contain address of the next node.
- The first node is assigned as the header and the address of the last node is said to null.
- **Singly linked list has only one pointer.**

Singly linked list



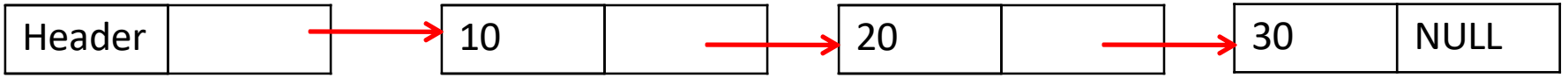


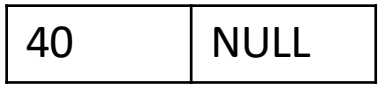
Operations on linked list

- Insertion
- Deletion
- Find
- Check whether the list is empty.
- Check islast element

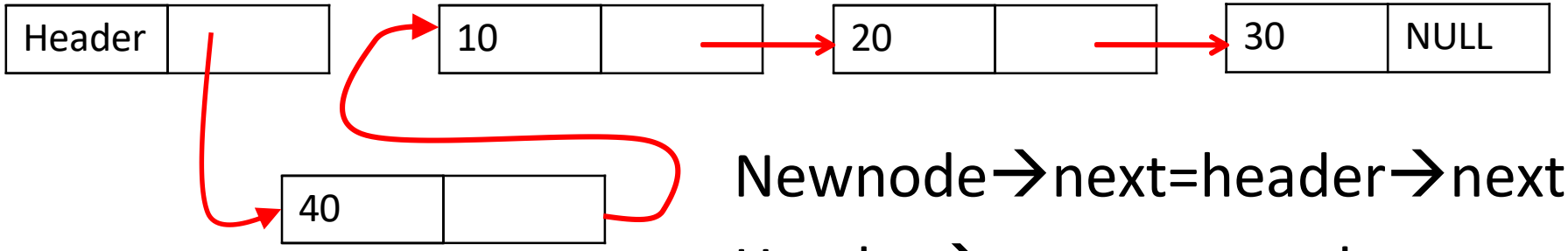
Insertion-3 ways

- **At the beginning of the list**





newnode



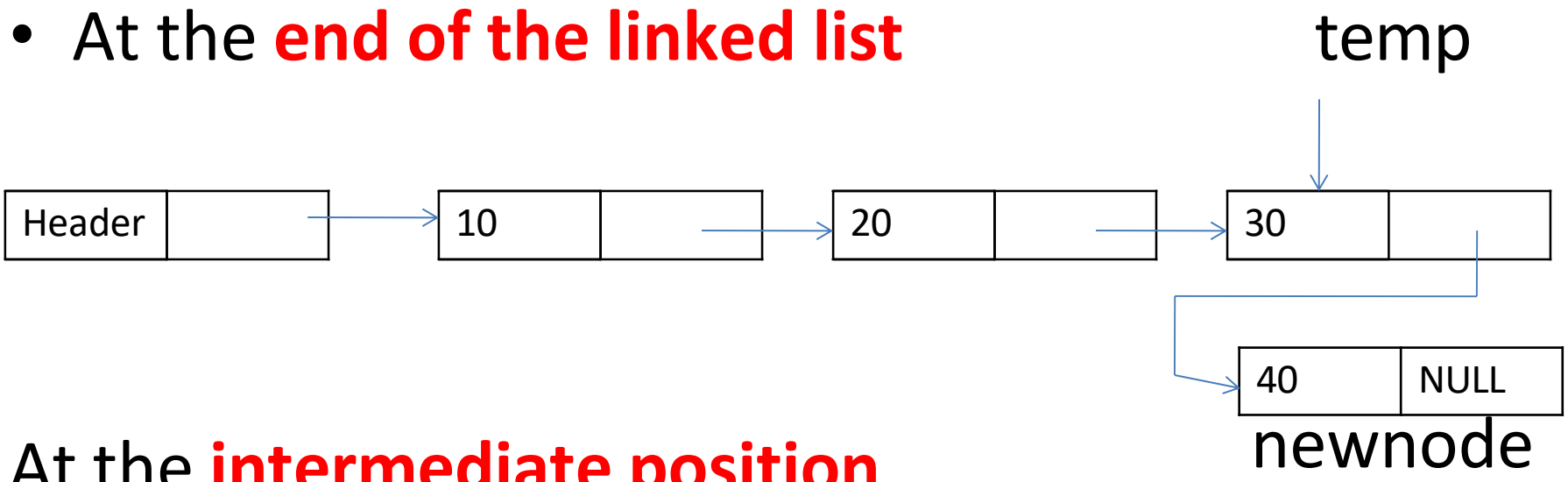
newnode

Newnode→next=header→next

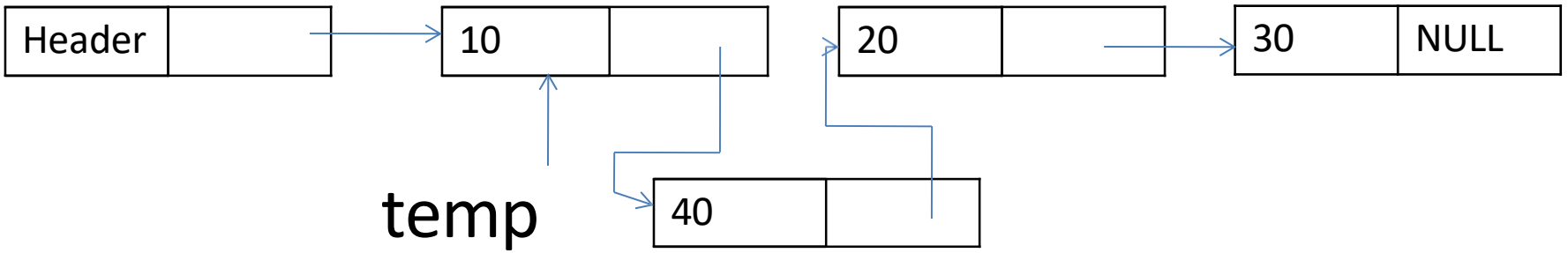
Header→next=newnode

Insertion

- At the **end of the linked list**



At the **intermediate position**



At the end

P→next=newnode;

At the intermediate position

Newnode->next=p→next

P→ next=newnode

- ROUTINE TO INSERT A NODE AFTER POSITION P

```
Void insert (int x,list L,position P)
{
        Newnode=malloc(sizeof(struct node));
        If(newnode!=NULL)
        {
                Newnode→data=x;
           Newnode→next=p→next;
                p→next=newnode;
        }
}
```

Routine to check whether list L is empty

```
Int isempty(list L)
{
return L -> next==NULL
}
```

Routine to check whether the current position is the last node in linked list.

```
Int islast(position P,list L)
{
Return P→next==NULL;
}
```

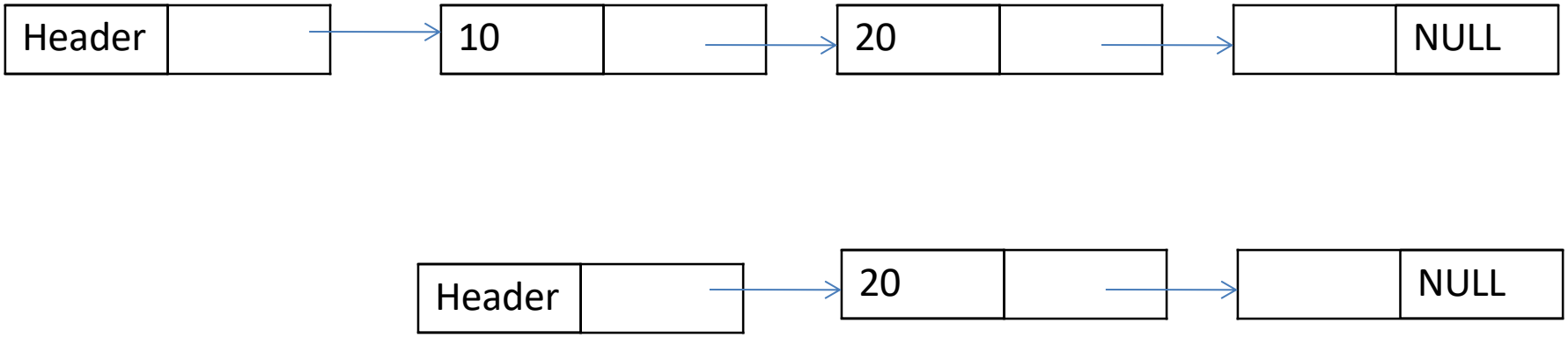
deletion

Memory is to be released for the node to be deleted

Deletion can be done in three different places

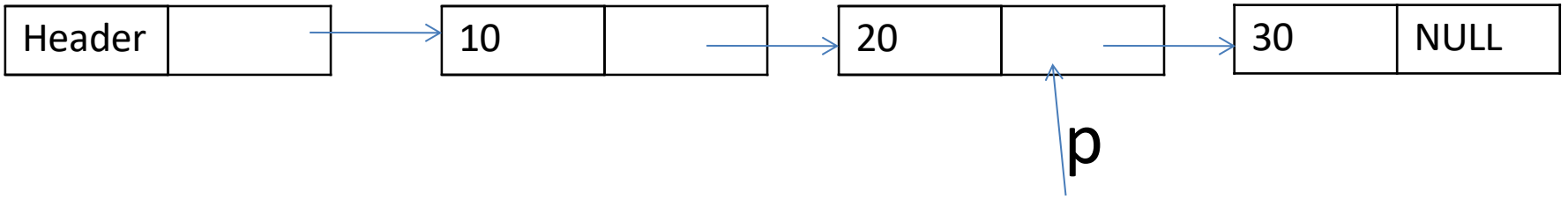
- Deleting the first node in the list
- Deleting the last node from the list
- Deleting the intermediate node from the list.

- Deleting the first node



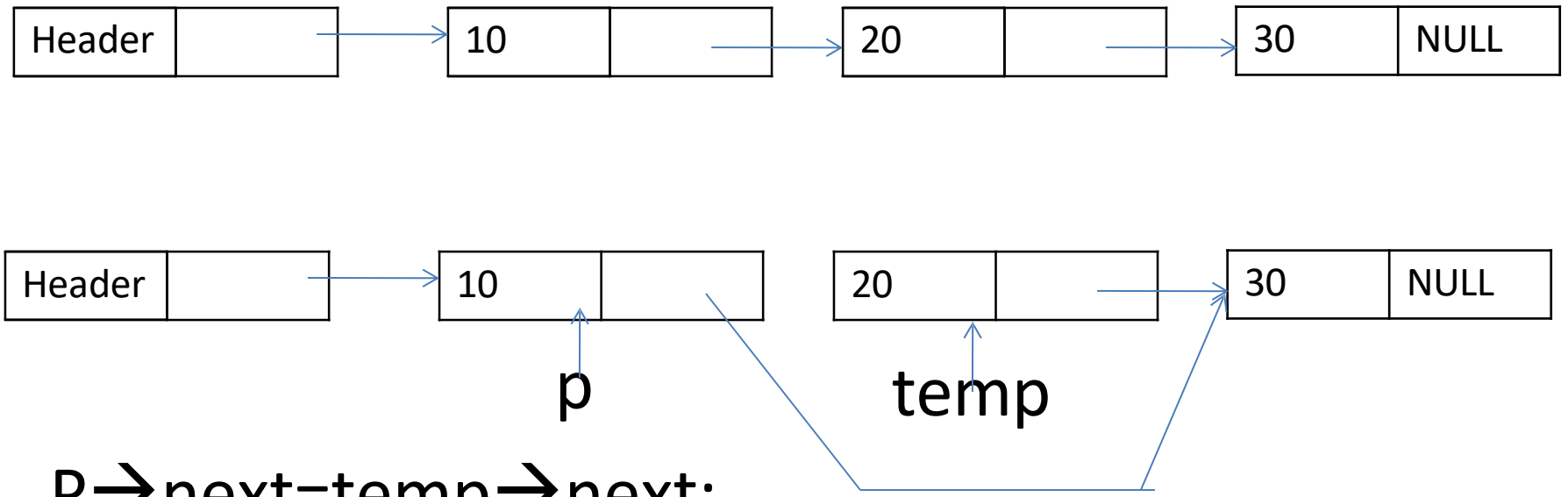
Header=Header→next

- Deleting last node



- Find previous(P)
- P→next=null

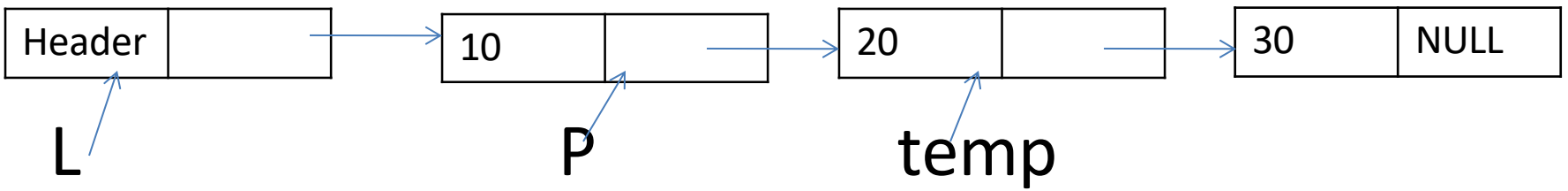
- Deleting intermediate node



P→next=temp→next;

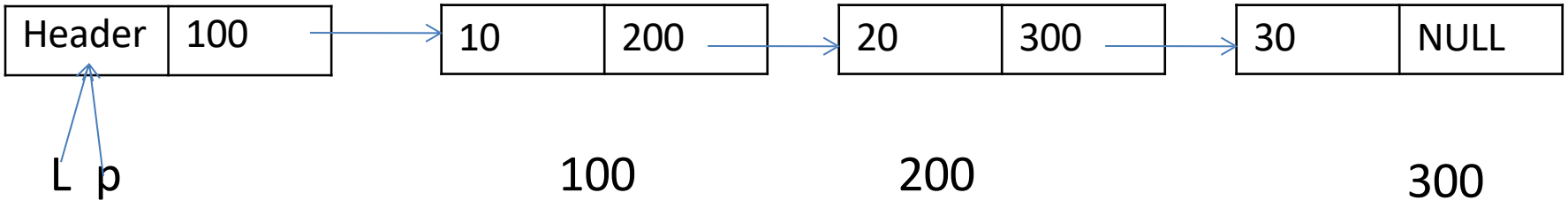
ROUTINE TO DELETE THE LIST

```
Void deletelist(Element type X,list L)
{
        Position P,temp;
        P=findprevious(X,L);
        if(!islast(P,L))
        {
                temp=P→next;
                P→next=temp→next;
                Free(temp);
                }
}
```



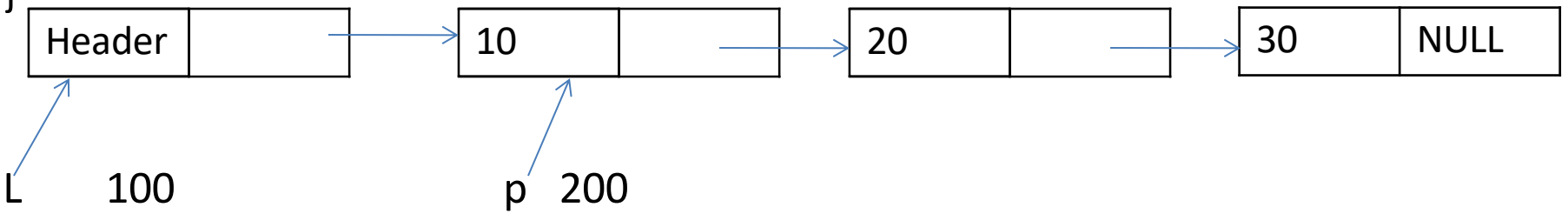
Routine to find the position of next element in list L

```
Position find(int x,list L)
{
Position P;
P=L
While(p→next!=NULL && P→data!=x)
{
P=p→next;
}
Return p→next;
}
```



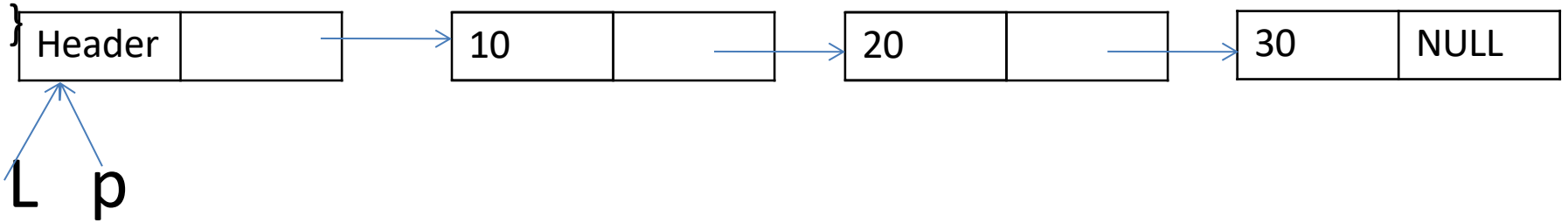
Routine to find the position of element x in list L

```
Position find(int x,list L)
{
Position P;
P=L→next;
While(p!=NULL && P→data!=x)
{
P=p→next;
}
Return p;
}
```



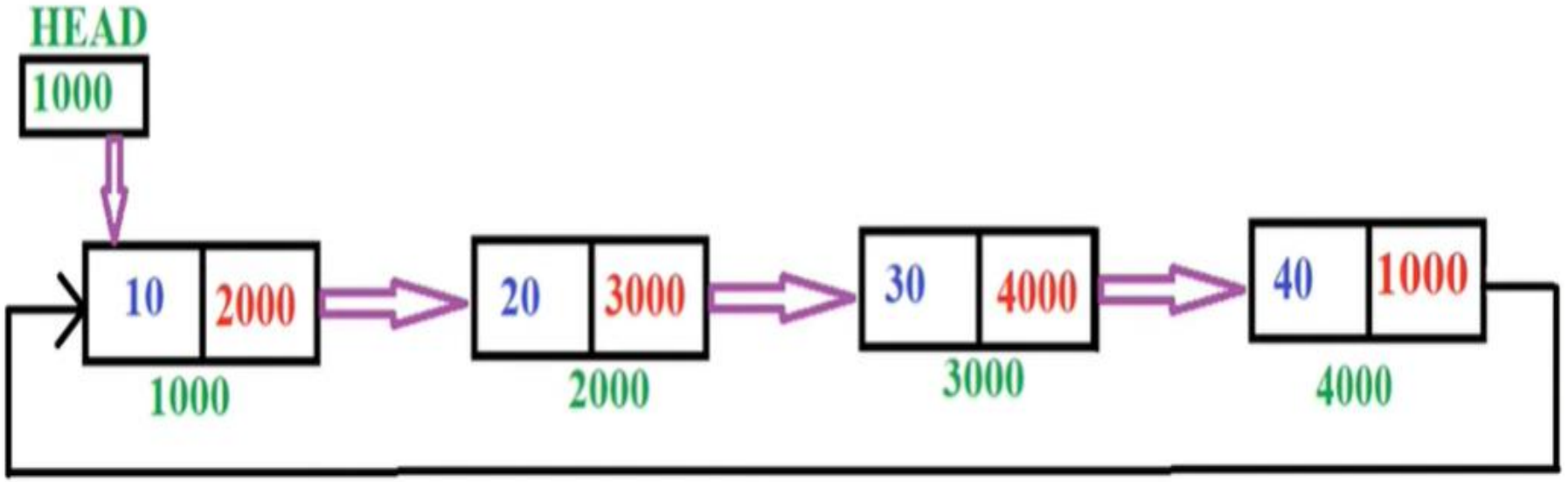
Routine to find the position of previous element in list L

```
Position findprevious(int x,list L)
{
Position P;
P=L;
While(p→next!=NULL && P→next→data!=x)
{
P=p→next;
}
Return p;
}
```



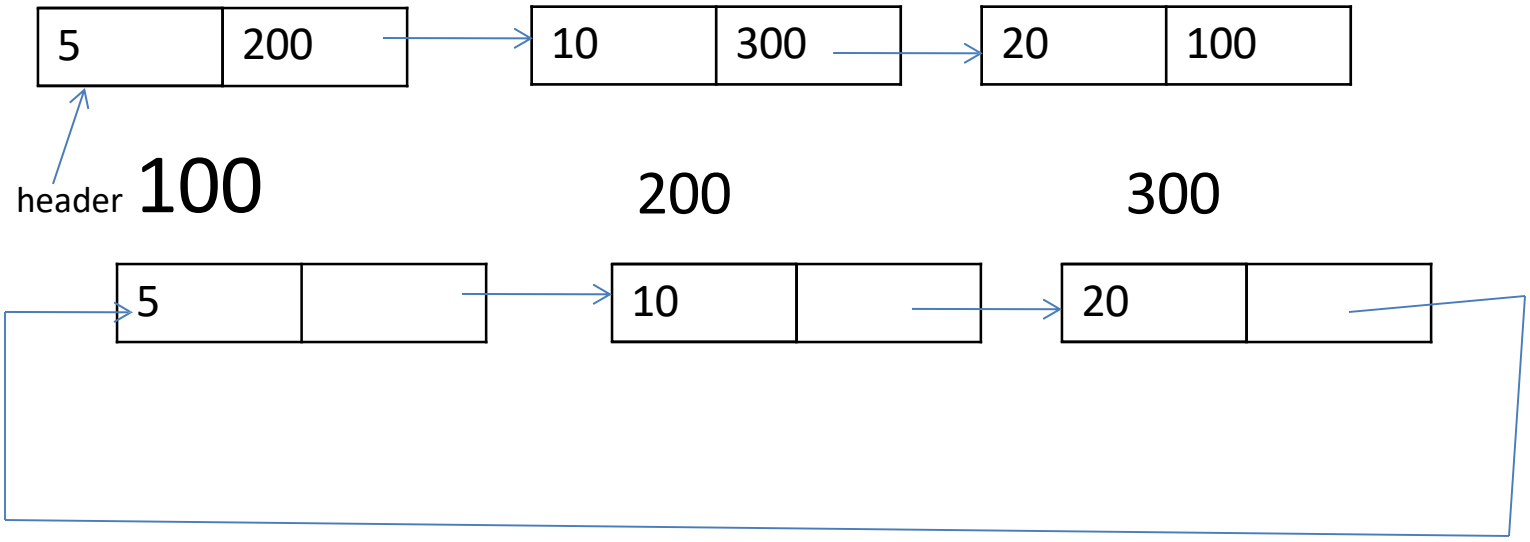
Circular Linked List

- A circular linked list is a sequence of elements in which every element has a link to its next element in the sequence and the last element has a link to the first element.



Circular linked list

- Circular linked list is a linear data structure.
- Circular linked list is similar to singly linked list except that the address field of last node contains address field of first node.

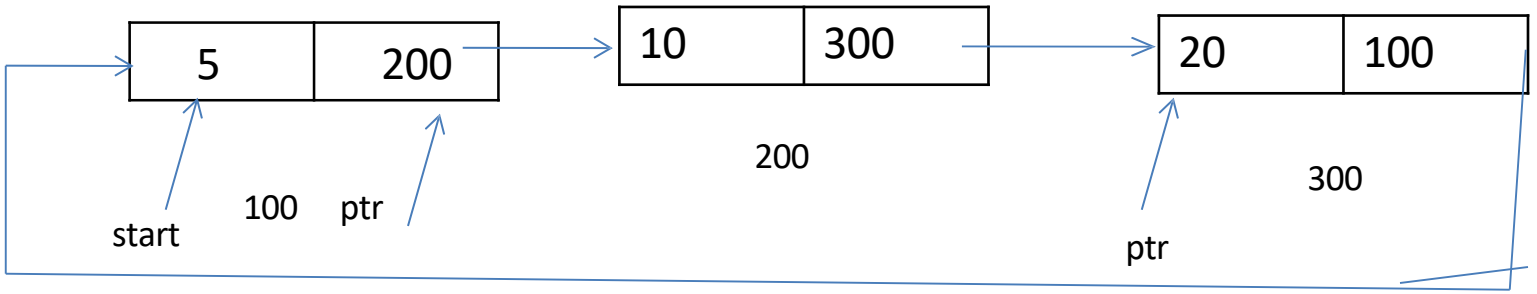


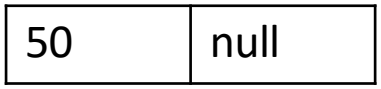
- Operations on circular linked list
  - creation
  - insertion
  - deletion
  - find
  - display

Insertion

1. Inserting At Beginning of the list
2. Inserting At End of the list
3. Inserting At Specific location in the list

Insertion First position

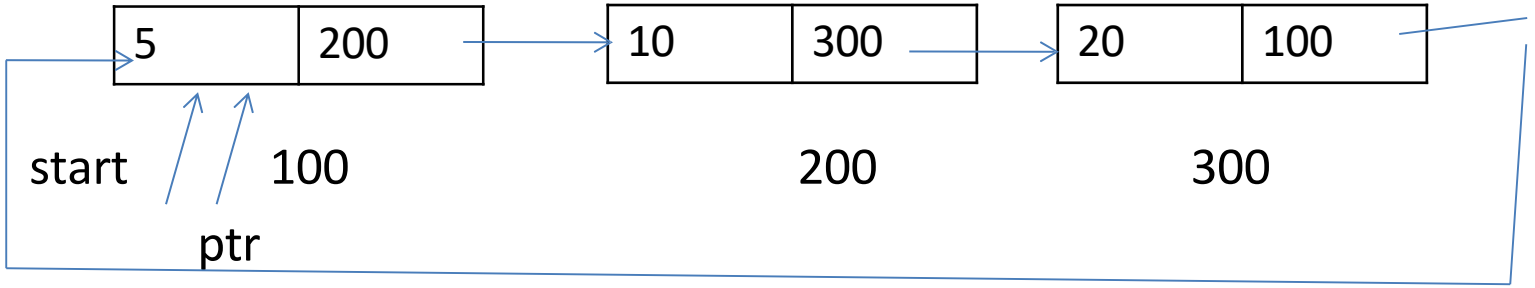




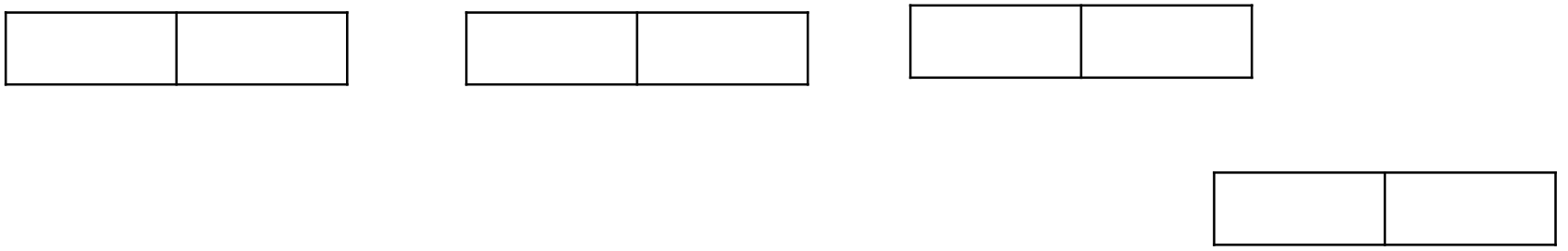
Newnode

1. Newnode -> data = item, Newnode -> Next = NULL
2. Newnode -> Next = Start
3. ptr = Start
4. while (ptr -> Next !=Start) ptr = ptr -> Next
5. Start = Newnode
6. Ptr -> Next = Start

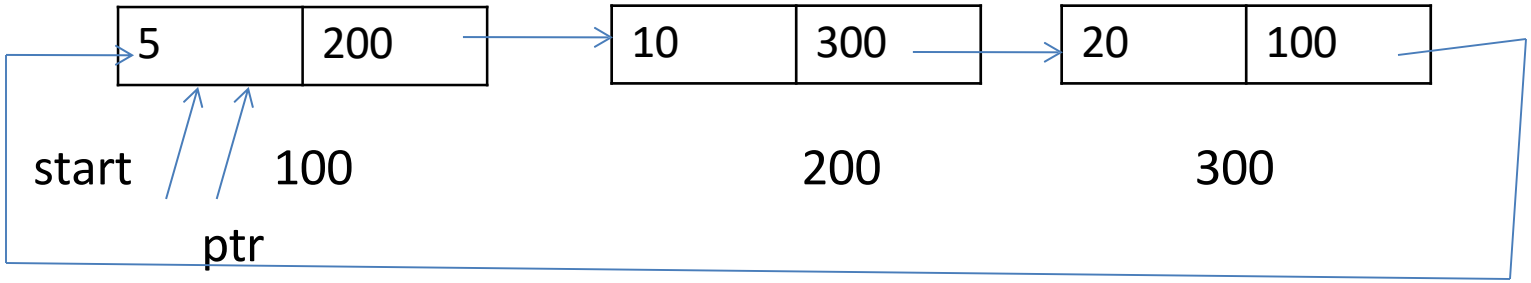
- Last position

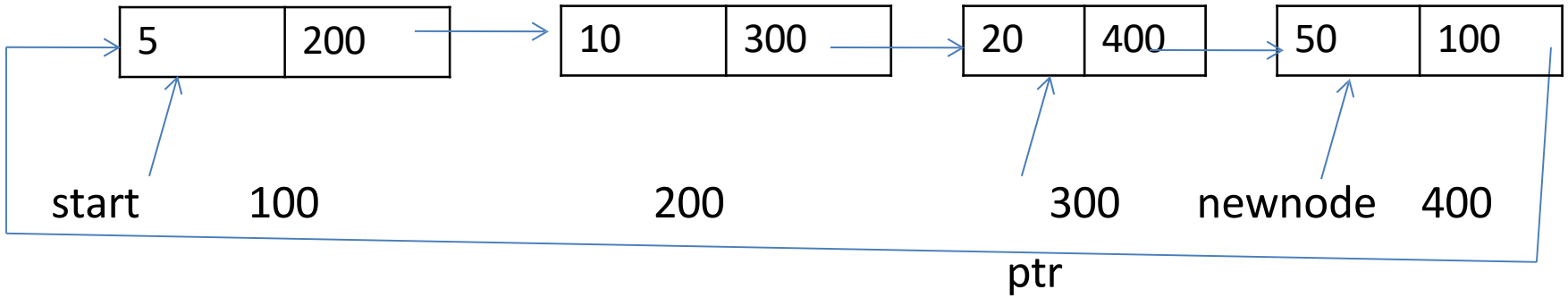


1. Newnode -> data = item, Newnode -> Next = NULL
2. ptr = Start
3. while (ptr -> Next !=Start) ptr = ptr -> Next
4. ptr -> Next = Newnode
5. Newnode -> Next = Start



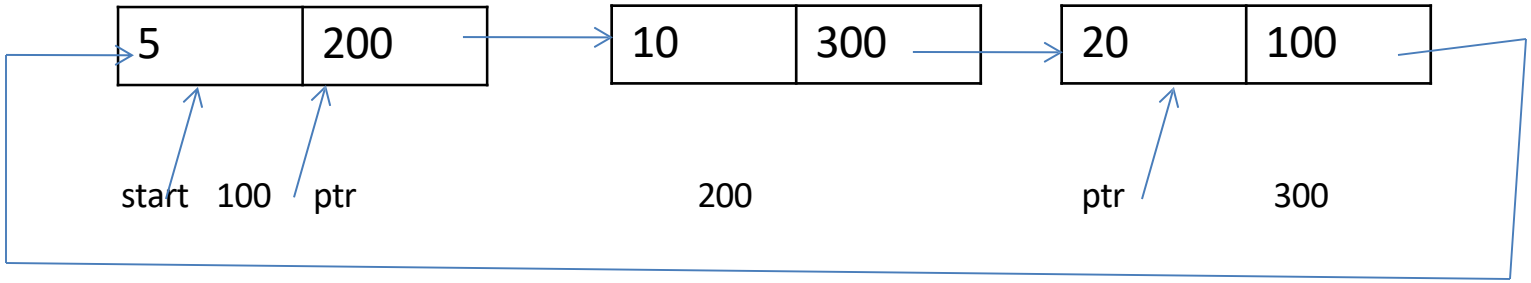
- Last position



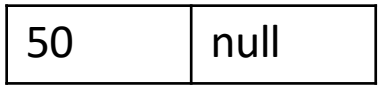


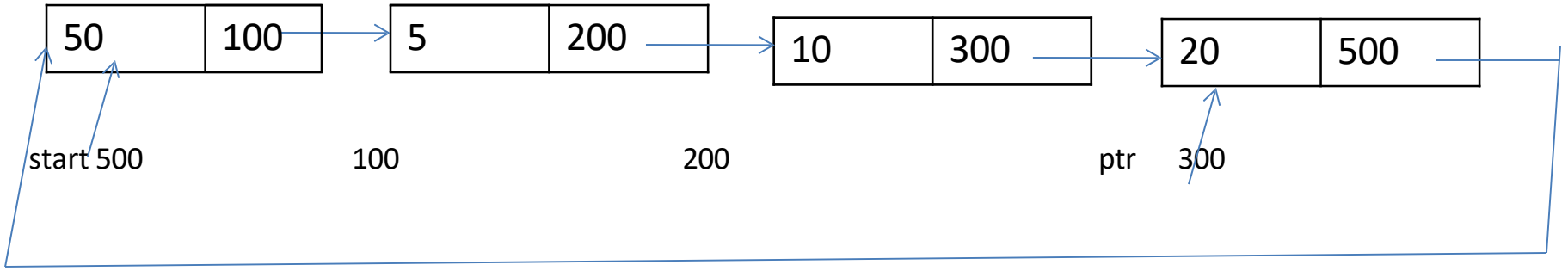
Insertion

First position



Newnode



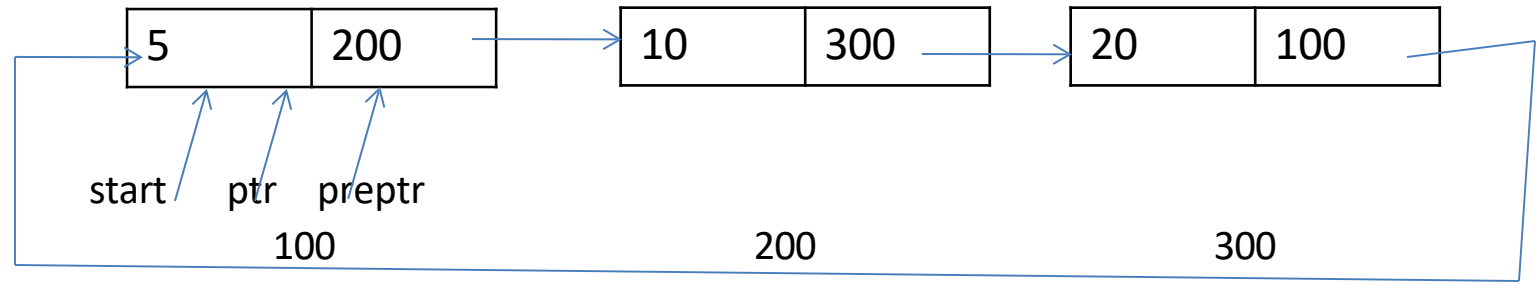


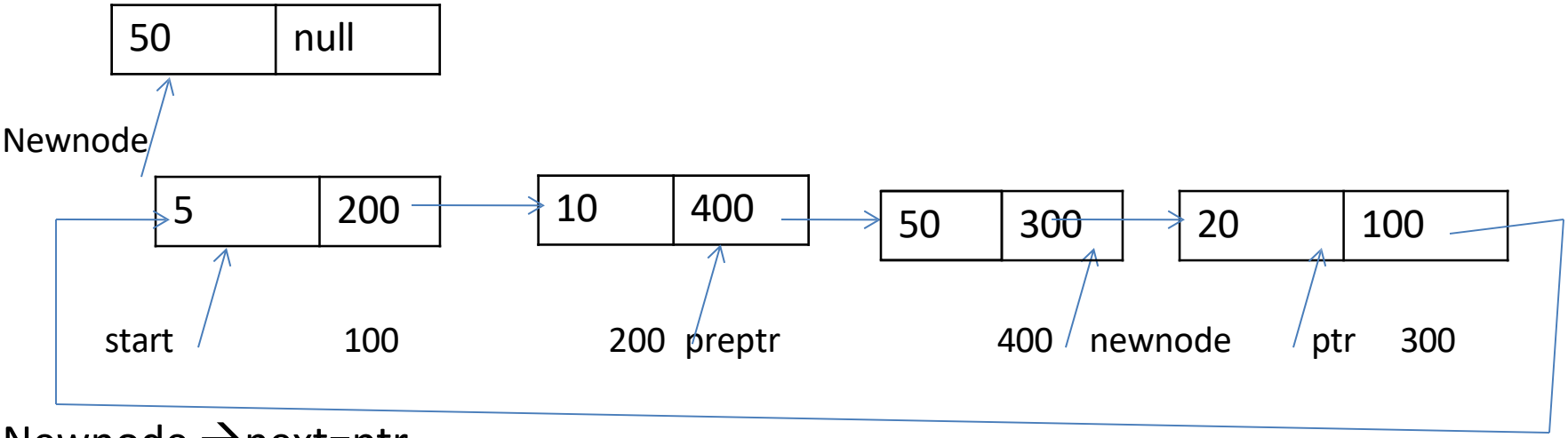
Newnode →next=start start=newnode ptr→next=newnode

Routine for insertion at the beginning

Step:1 if avail== null
write overflow
go to step 11
Step:2 set newnode=avail
Step:3 set avail=avail→next
Step:4 set newnode→data=val
Step:5 set ptr=start
Step:6repeat step 7 while(ptr→next !=start)
Step:7 ptr=ptr→next
end of loop
Step:8 set newnode→next=start
Step:9 set ptr→next=newnode
Step:10 set start = newnode
Step:11 exit

- Intermediate position





Newnode →next=ptr

preptr→next =newnode

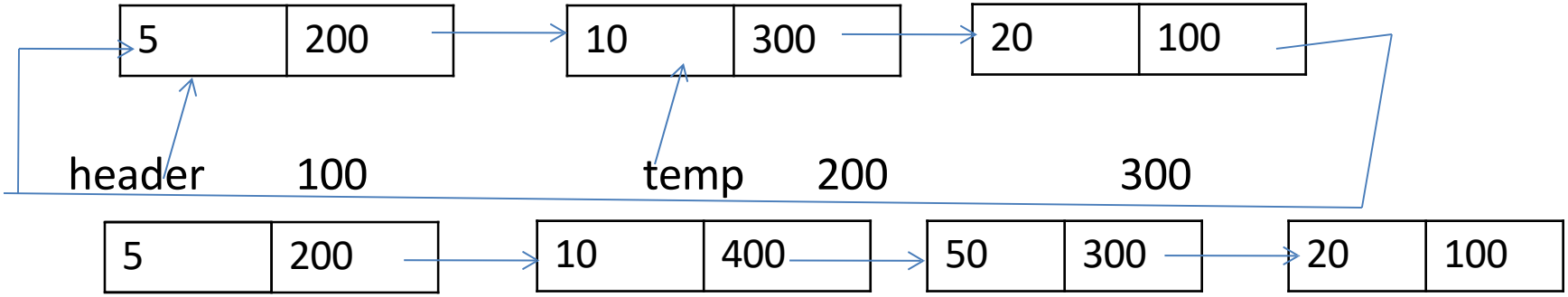
Routine for insertion at the intermediate position

Step:1 if avail== null
write overflow
go to step 13
Step:2 set newnode=avail
Step:3 set avail=avail→next
Step:4 set newnode→data=val
Step:5 set ptr=start
Step:6 set preptr=ptr
Step:7 repeat step 8 and 9 while(preptr→data !=num)
Step:8 set preptr=ptr
Step:9 set ptr=ptr→next
end of loop
Step:11 set preptr→next=newnode
Step:12 set newnode→next= ptr
Step:13exit

Routine for insertion at the End

Step:1 if avail== null
write overflow
go to step 10
Step:2 set newnode=avail
Step:3 set avail=avail→next
Step:4 set newnode→data=val
Step:5 set ptr=start
Step:6repeat step 7 while(ptr→next !=start)
Step:7 ptr=ptr→next
end of loop
Step:8 set newnode→next=start
Step:9 set ptr→next=newnode
Step:10 exit

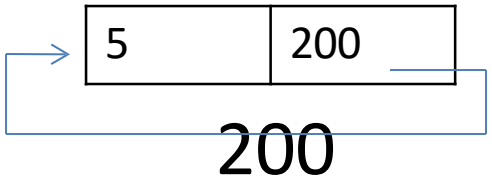
```
Insert()
{
Int val,key;
New =getnode();
Printf("enter the data");
Scanf("%d",&val);
New →data=val;
Printf("after which element u want to insert");
Scanf("%d",&key);
Temp=search(key);
New→next=temp→next;
Temp→next=new;
}
```

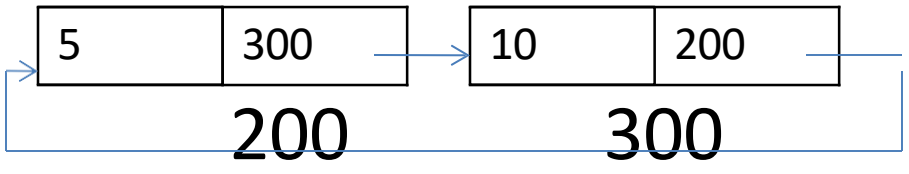


Creation of CLL

```
Create()
{
Int x=1;
Int  val;
Char  b;
Do
{
Newnode=malloc(sizeof(struct
node));
Printf("enter a data");
Scanf("%d",&val);
Newnode→data=val;
If(x==1)
{
start=newnode;
start→next=start;
X=0;
}
```

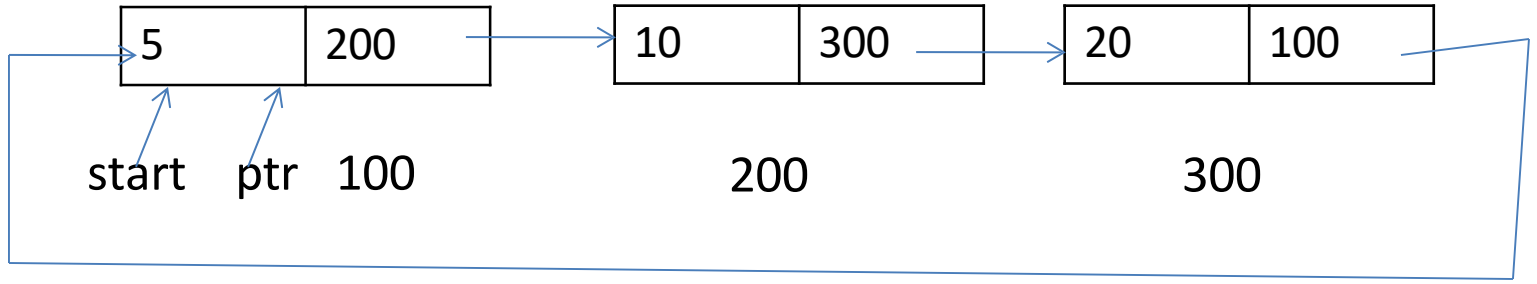
```
Else
{
ptr=start;
While(ptr→next!=start)
{
ptr=ptr→next;
}
ptr→next=newnode;
Newnode→next=start;
}
Printf(do you want to
     continue");
b=getchar();
}while(b=='y');
}
```





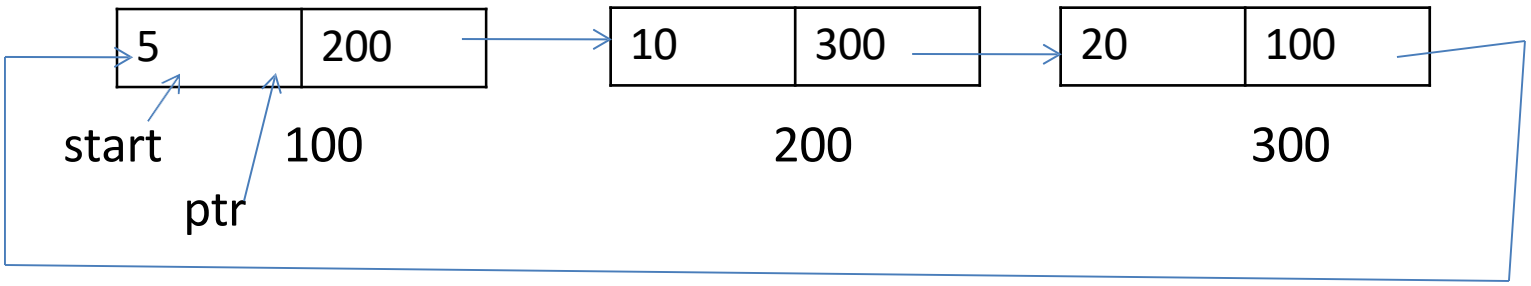
Searching in CLL

```
SEARCH(int key)
{
Int found=0;
ptr=start;
While(ptr→next!=start && found==0)
{
If(ptr→data!=key)
{
ptr=ptr→next;
}
Else
{
Found=1;
}
}
If(ptr→data==key)
{
Found=1;
}
If (found)
{
Return(ptr);
}
Else
{
Return(NULL);
}
}
```



Display of CLL

```
Display()
{
ptr=start;
If(ptr==null)
{
Printf("list is empty");
}
Else
{
While(ptr→next!=start)
{
Printf("%d",ptr→data);
ptr=ptr→next;
}
Printf ("%d",ptr→data);
}
}
```

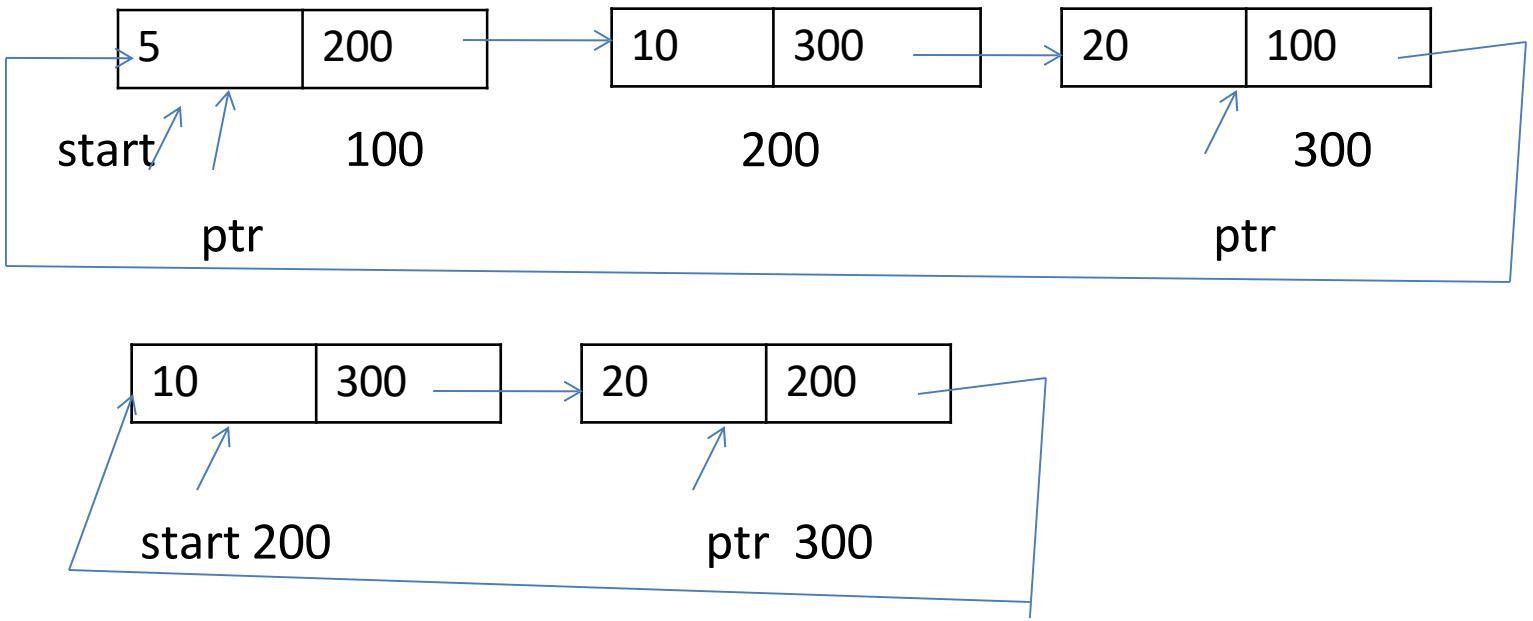


deletion

Deletion can be done in three different places

- First position
- Intermediate position
- Last position

- First position



Routine for delete at the beginning

Step:1 if start == null

write underflow

go to step 8

Step:2 set ptr=start

Step:3repeat step 4 while(ptr→next !=start)

Step:4 ptr=ptr→next

end of loop

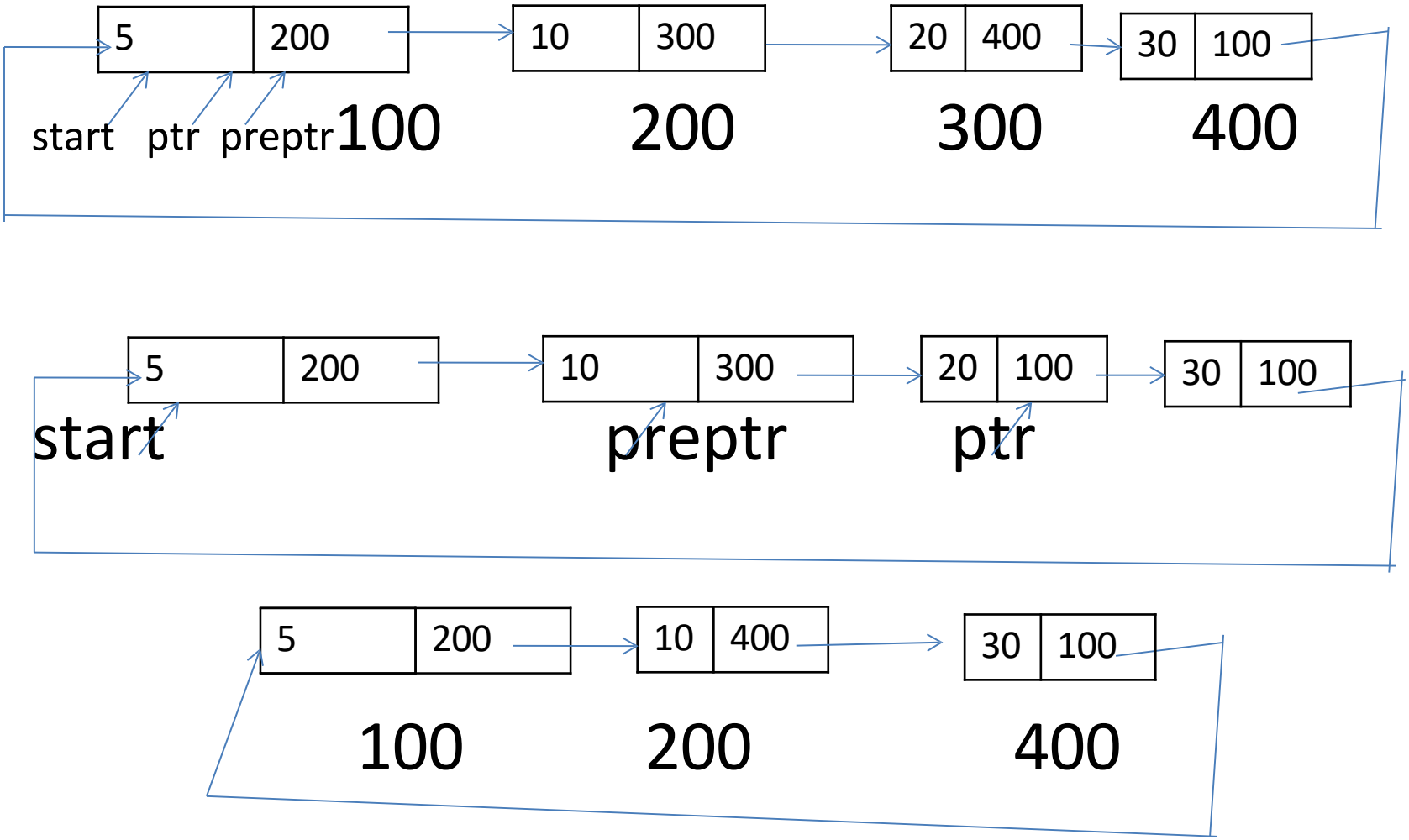
Step:5 set ptr→next=start→next

Step:6 free start

Step:7 set start = ptr→next

Step:8 exit

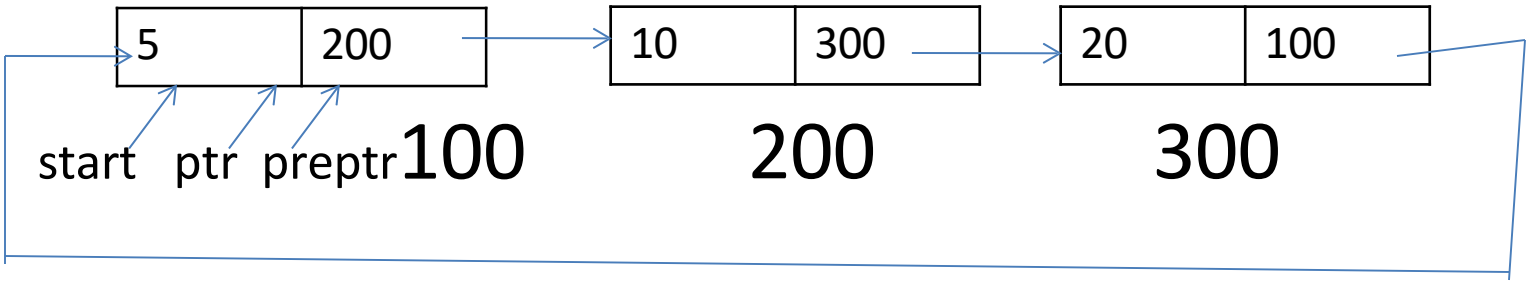
Intermediate position

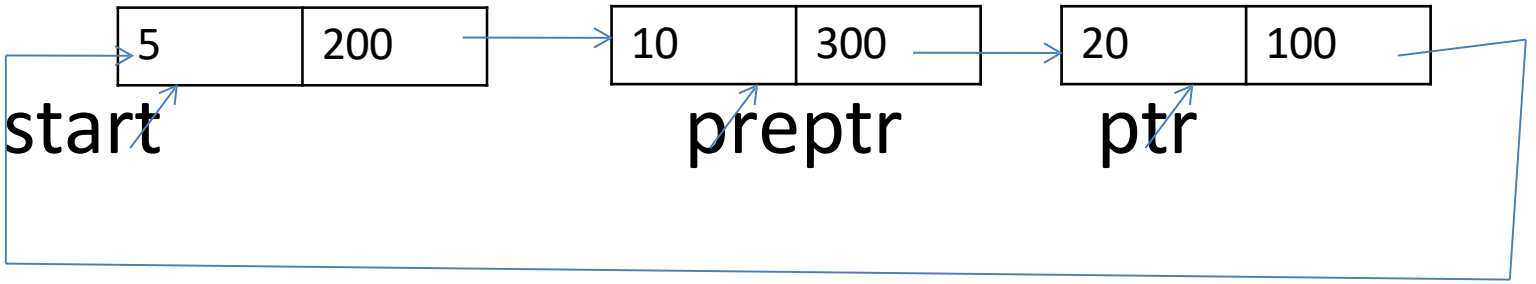


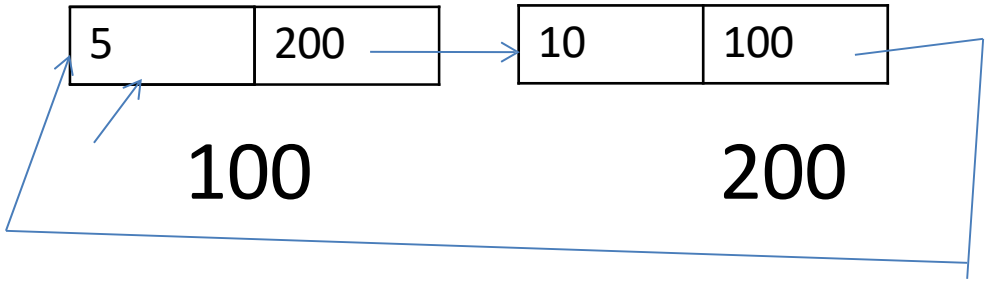
Routine for delete at the intermediate position

Step:1 if start == null
write underflow
go to step 9
end of if
Step:2 set ptr=start
Step:3 set preptr=ptr
Step:4 repeat step 5 and 6 while(preptr→data !=num)
Step:5 set preptr=ptr
Step:6 ptr=ptr→next
end of loop
Step:7 set preptr→next=ptr→next
Step:8 free ptr
Step:9 exit

Last position







Routine for delete at the end

Step:1 if start == null

write underflow

go to step 8

end of if

Step:2 set ptr=start

Step:3repeat step 4 and 5 while(ptr→next !=start)

Step:4 set preptr=ptr

Step:5 ptr=ptr→next

end of loop

Step:6 set preptr→next=start

Step:7 free ptr

Step:8 exit

Doubly linked list

Doubly linked list is a linear data structure.

DLL is a two way linked list with collection of node.

Each node contains 3 fields

-previous address

-data

-next address

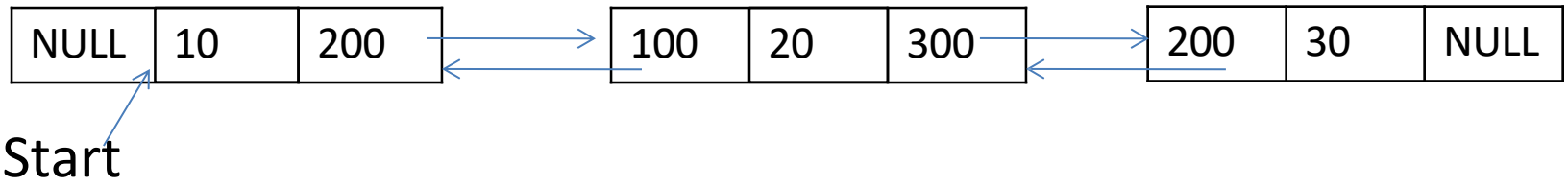
Previous address	data	Next address

It consist of two pointers

-next pointer

-previous pointer

Doubly linked list



Operations on doubly linked list

Creation

Insertion

Deletion

Display

creation

Creating a node in DLL

```
Struct node
{
Int data;
Struct node *next;
Struct node *prev;
};
```

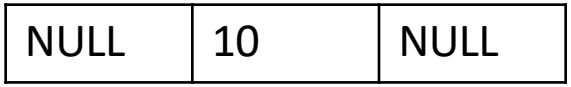
Newnode=malloc(sizeof(struct node);

Newnode→prev=null;

Newnode→next=null;

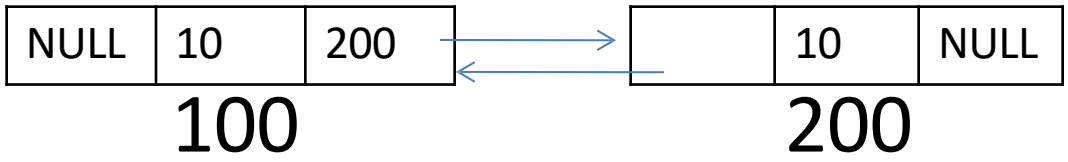
Newnode→data=10;

start=newnode;



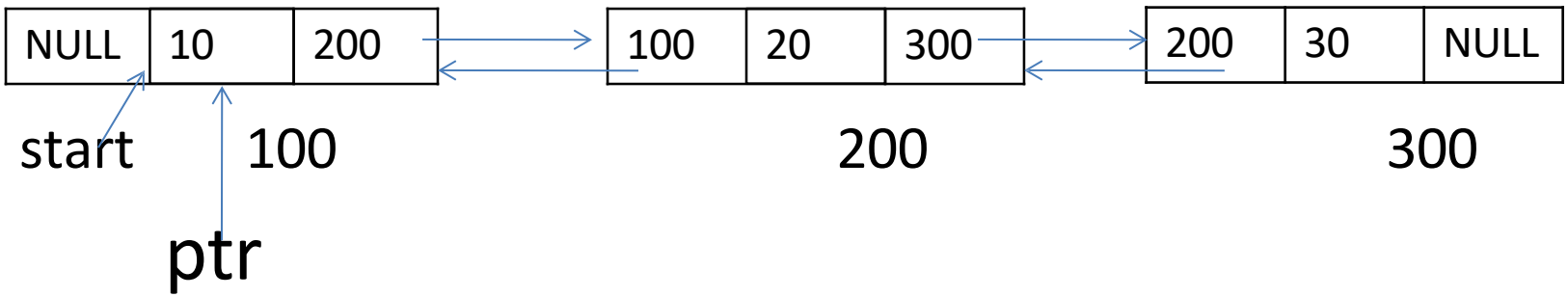
start→next=newnode;

Newnode→prev=start;



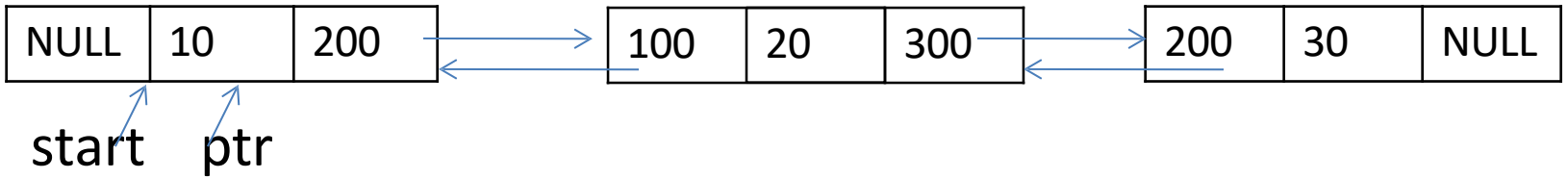
- Display

```
ptr=start;
While(ptr→next!=NULL)
{
Printf("%d",ptr→data);
ptr=ptr→next;
}
Printf("%d",ptr→data);
```



- **Searching**

```
ptr=start;
While(ptr→next!=NULL)
{
If(ptr→data==key)
Printf("key is found");
ptr=ptr→next;
}
If(ptr→data==key)
Printf("key is found");
}
```



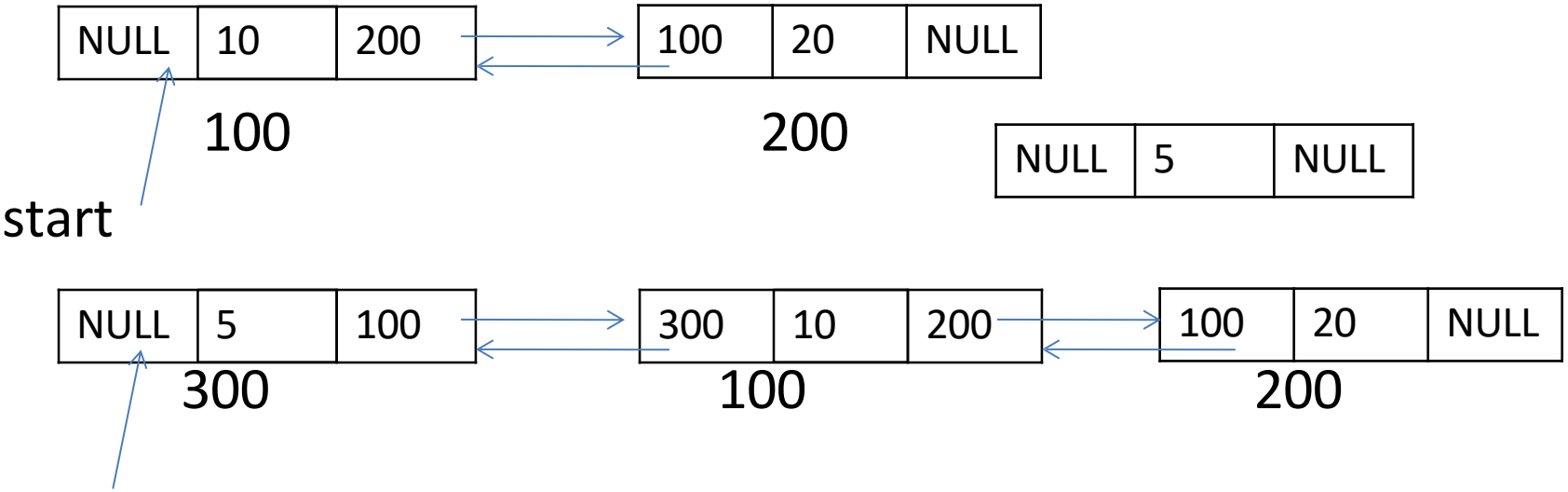
insertion

First position

Intermediate position

Last position

First position:



Routine for insertion at the beginning

Step:1 if avail== null

write overflow

go to step 9

end if

Step:2 set newnode=avail

Step:3 set avail=avail→next

Step:4 set newnode→data=val

Step:5 set newnode→prev=null

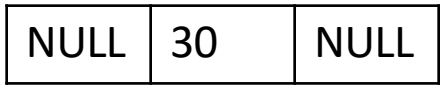
Step:6 set newnode→next=start

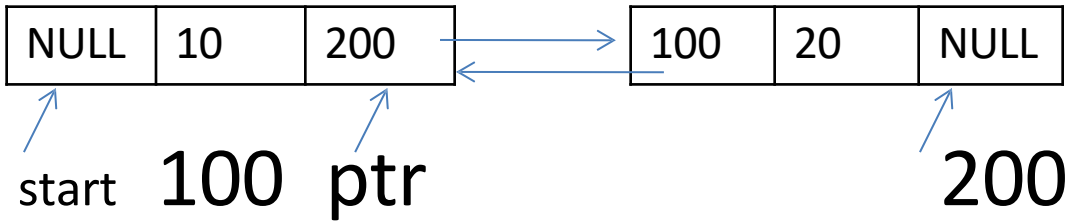
Step:7 set start→prev=newnode

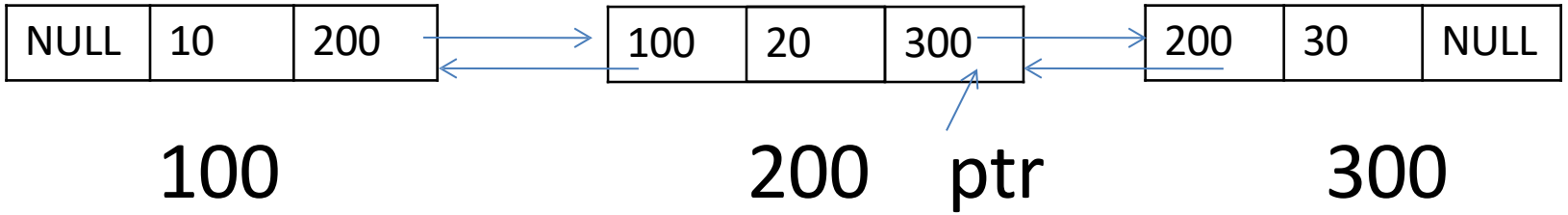
Step:8 set start = newnode

Step:9 exit

- Last position



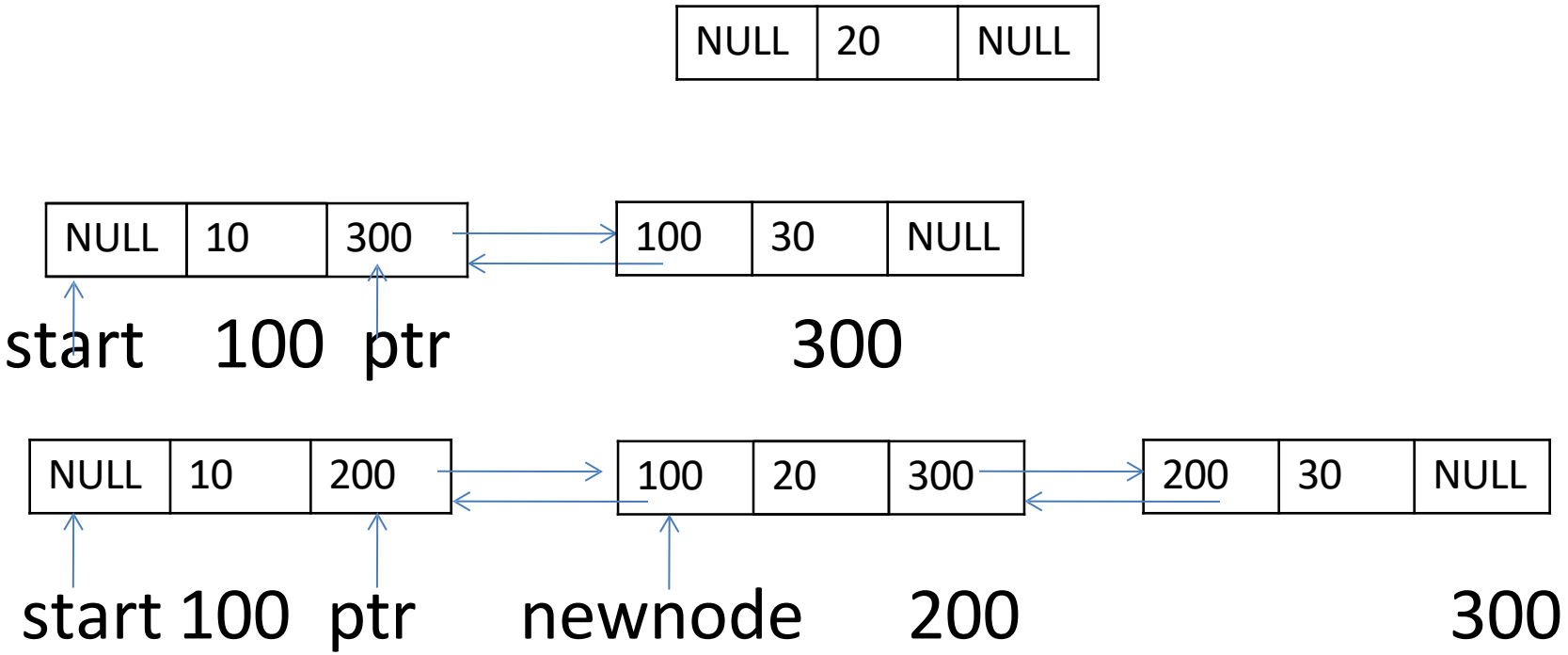




Routine for insertion at the end

Step:1 if avail== null
write overflow
go to step 11
end if
Step:2 set newnode=avail
Step:3 set avail=avail→next
Step:4 set newnode→data=val
Step:5 set newnode→next=null
Step:6 set ptr=start
Step:7 repeat step 8 while(ptr→next!=NULL)
Step:8 set ptr=ptr→next
end loop
Step:9 set ptr→next=newnode
Step:10 set newnode→prev=ptr
Step:11 exit

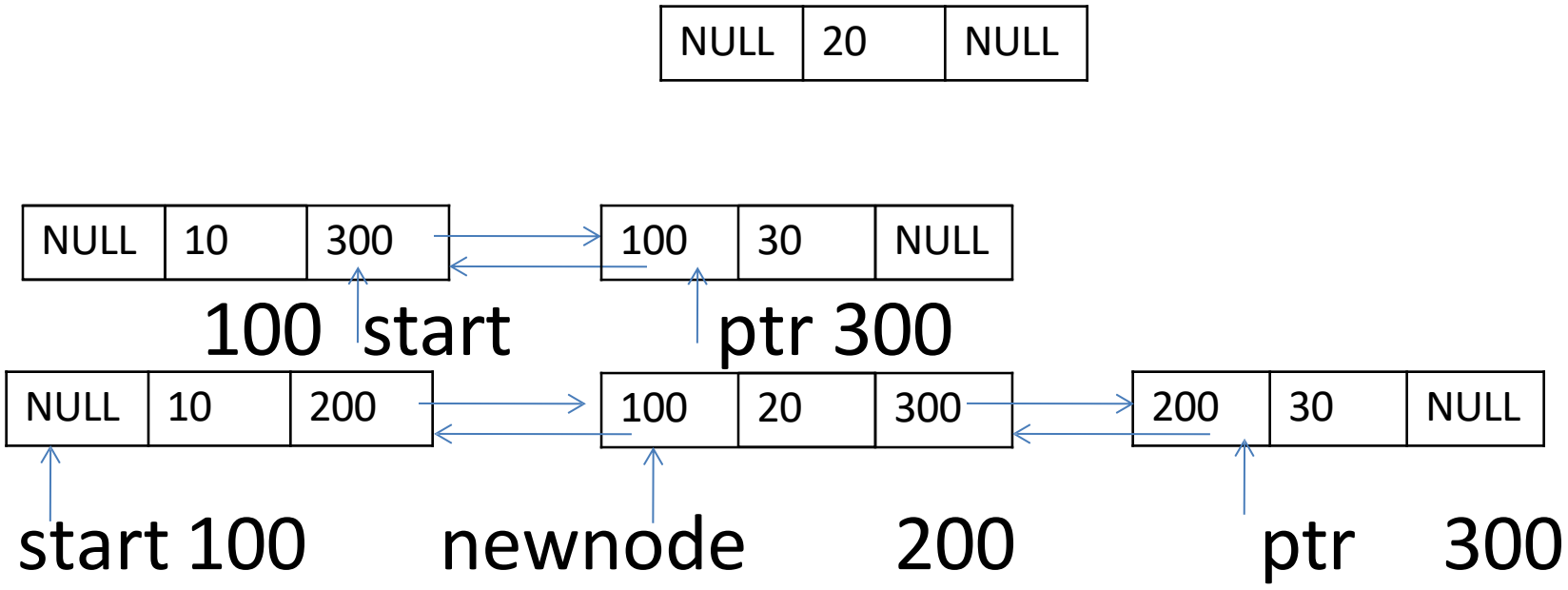
INTERMEDIATE POSITION



Routine to insert a newnode after a given node

Step:1 if avail== null
write overflow
go to step 12
end if
Step:2 set newnode=avail
Step:3 set avail=avail→next
Step:4 set newnode→data=val
Step:5 set ptr=start
Step:6 repeat step 7 while(ptr→data!=num)
Step:7 set ptr=ptr→next
end loop
Step:8 set newnode→next=ptr→next
Step:9 set newnode→prev=ptr
Step:10 set ptr→next=newnode
Step:11 set ptr→next→prev=newnode
Step:12 exit

INTERMEDIATE POSITION



Routine to insert a newnode before a given node

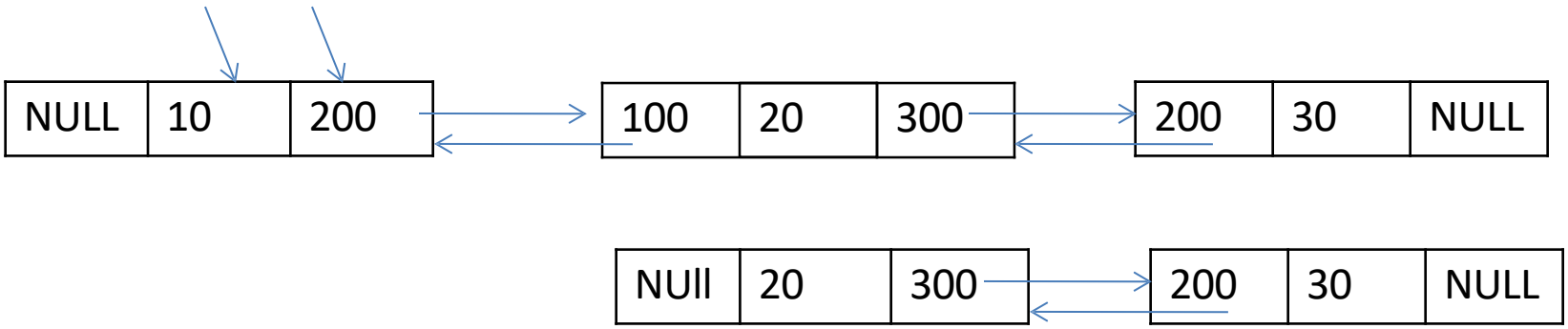
Step:1 if avail== null
write overflow
go to step 12
end if
Step:2 set newnode=avail
Step:3 set avail=avail→next
Step:4 set newnode→data=val
Step:5 set ptr=start
Step:6 repeat step 7 while(ptr→data!=num)
Step:7 set ptr=ptr→next
end loop
Step:8 set newnode→next=ptr
Step:9 set newnode→prev=ptr→prev
Step:10 set ptr→prev=newnode
Step:11 set ptr→prev→next=newnode
Step:12 exit

Deletion

- Deleting first node
- Deleting last node
- Deleting a node after a given node
- Deleting a node before a given node

deletion

Front position



Step:1 if start= null

write underflow

goto step 6

endif

Step:2 set ptr=start

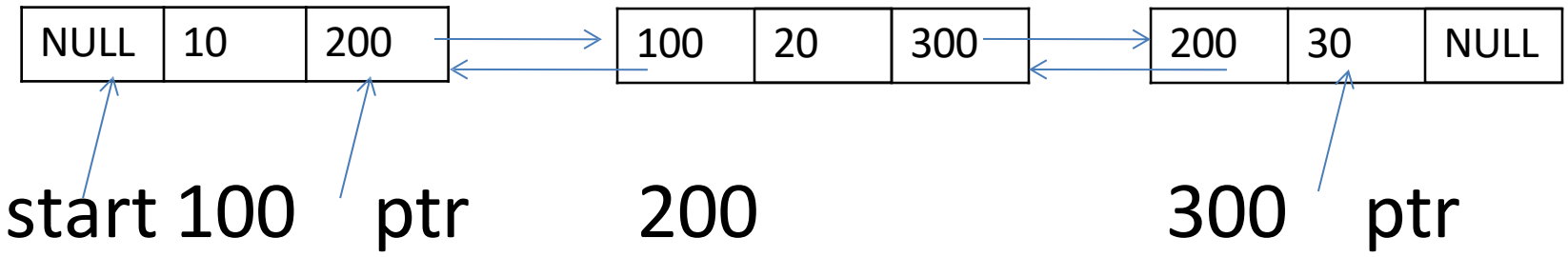
Step:3 set start=start→next

Step:4 set start→prev=null

Step:5 free ptr

Step:6 exit

- Last position



Step:1 if start= null

write underflow

goto step 7

endif

Step:2 set ptr=start

Step:3 repeat step 4 while ptr→next!=null

Step:4 set ptr=ptr→next

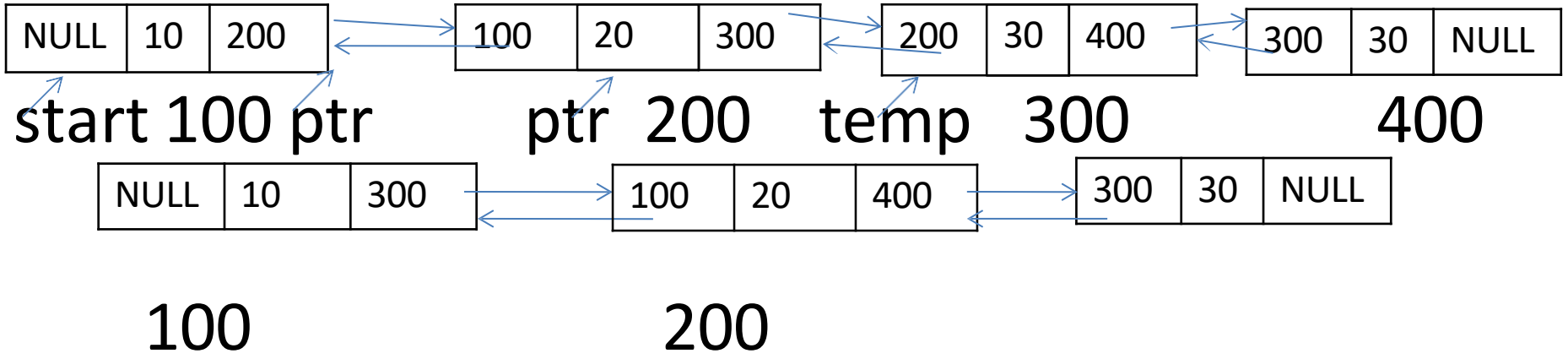
end loop

Step:5 set ptr→prev→next=null

Step:6 free ptr

Step:7 exit

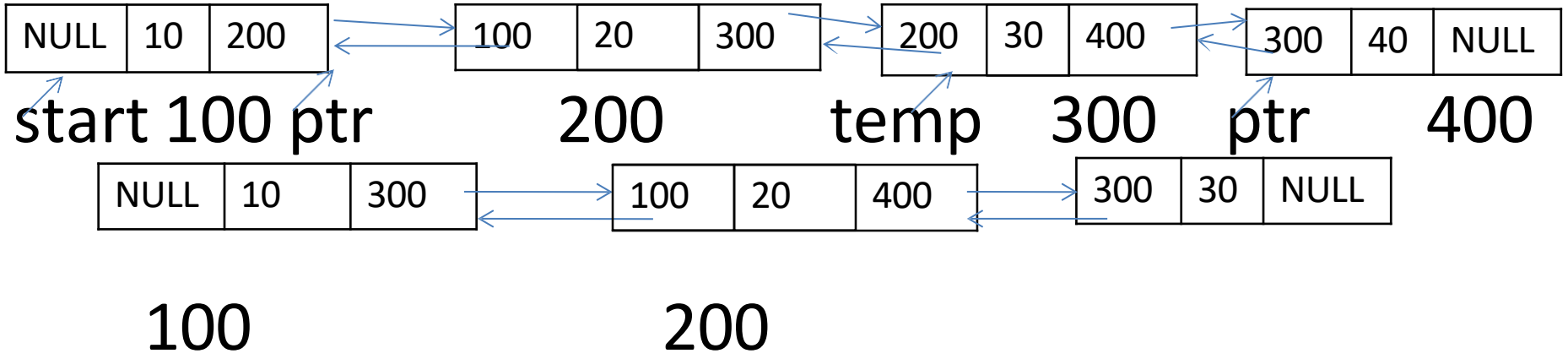
Deleting a node after a given node



Deleting a node after a given node

Step:1 if start= null
write underflow
goto step 9
endif
Step:2 set ptr=start
Step:3 repeat step 4 while ptr→data!=num
Step:4 set ptr=ptr→next
end loop
Step:5 set temp=ptr→next
Step:6 set ptr→next=temp→next
Step:7 set temp→next→prev=ptr
Step:8 free temp
Step:9 exit

- **Deleting a node before a given node**



Deleting a node before a given node

Step:1 if start= null

write underflow

goto step 9

endif

Step:2 set ptr=start

Step:3 repeat step 4 while ptr→data!=num

Step:4 set ptr=ptr→next

end loop

Step:5 set temp=ptr→prev

Step:6 set temp→prev→next=ptr

Step:7 set ptr→prev=temp→prev

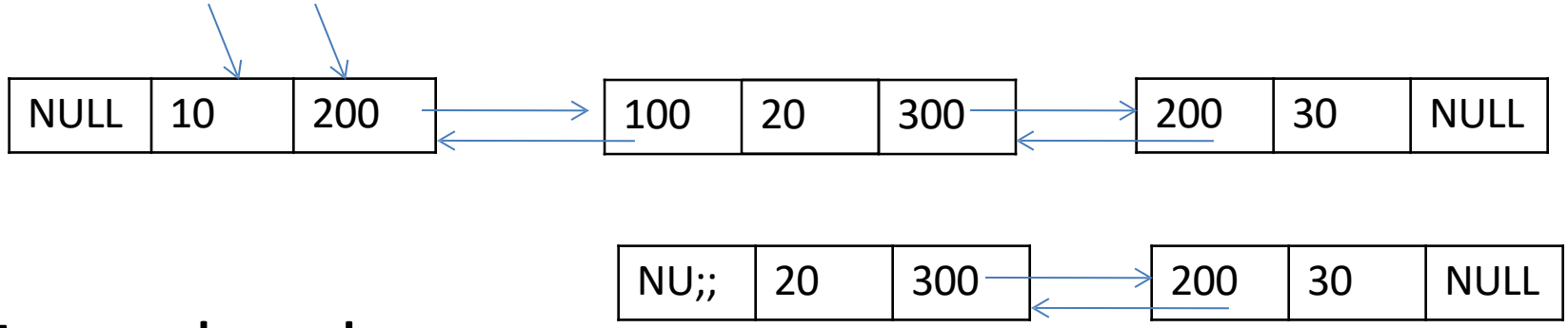
Step:8 free temp

Step:9 exit

deletion

Front position

head temp



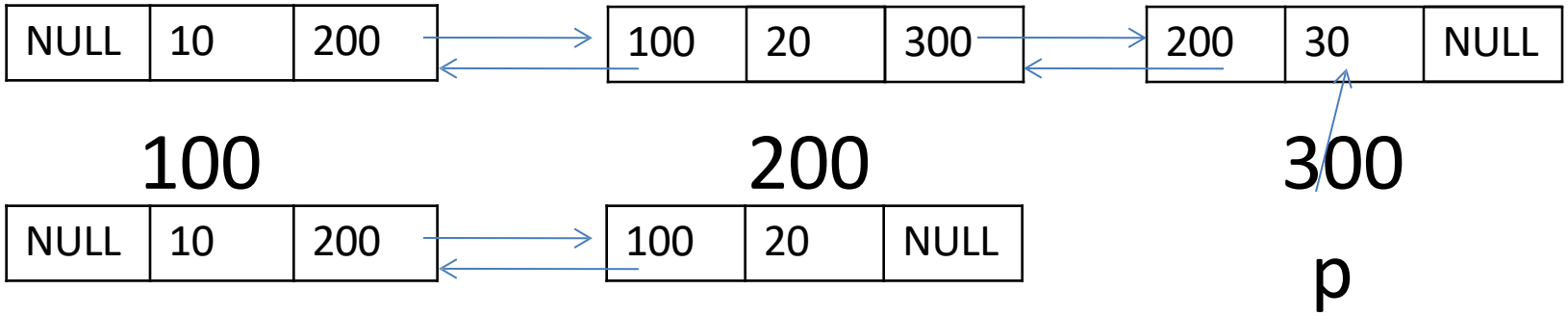
Temp=head;

Head→next→prev=NULL;

Head=temp→next;

Free(temp);

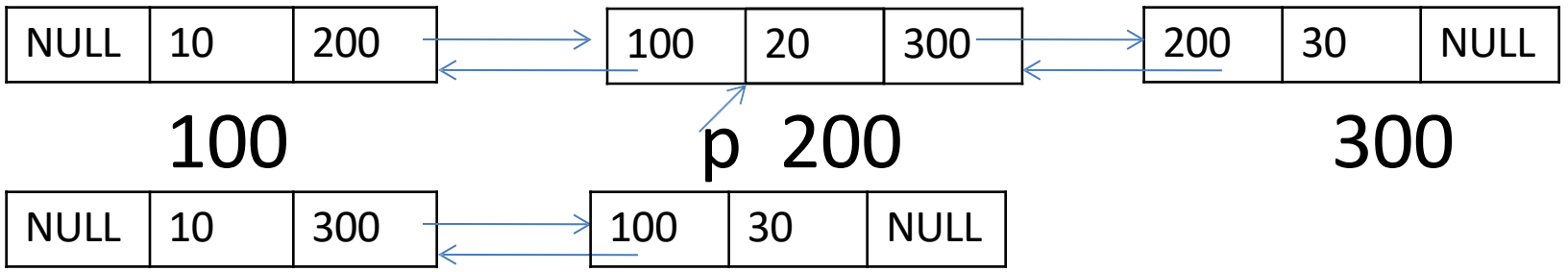
- Last position



p→prev→next=NULL;

Free(p);

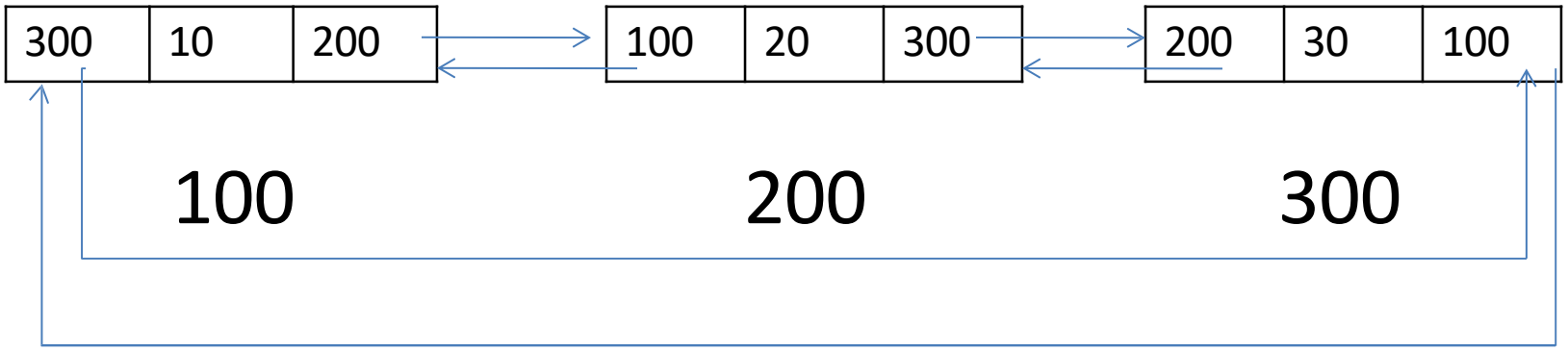
- Intermediate position



Head→next=p→next;

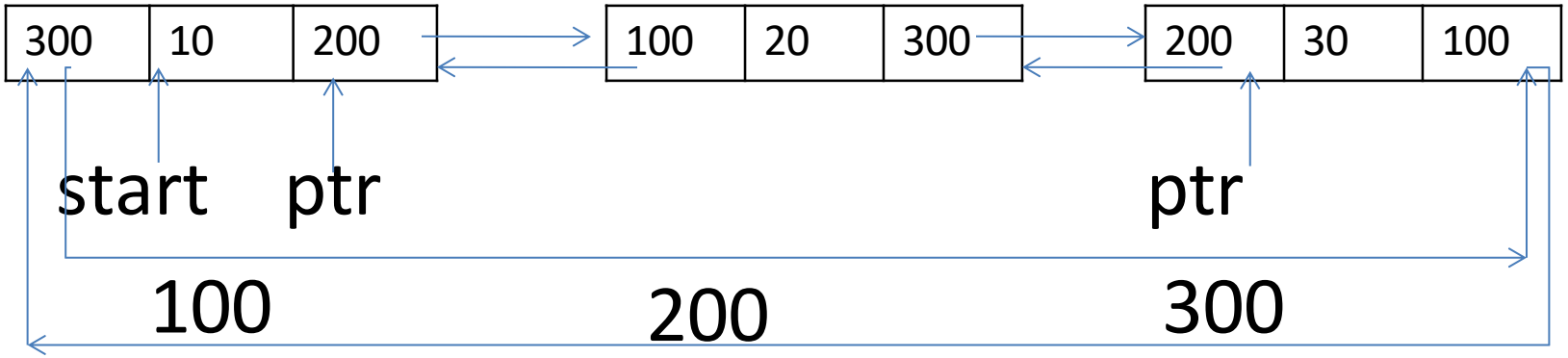
P→next→prev=head;

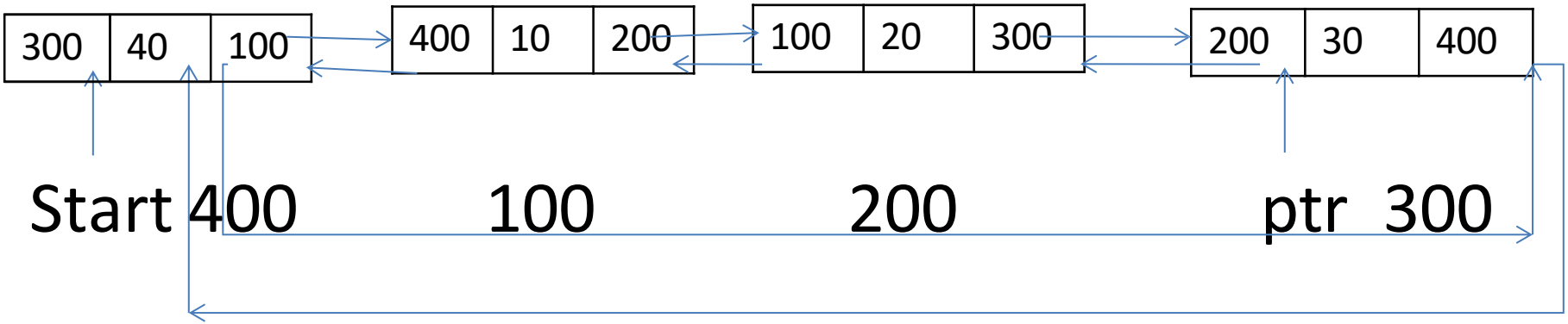
- **Circular doubly linked list**
  - it is a circular two way linked list
  - It contains a pointer to the next as well as the previous node in the sequence
  - The next field of the last node stores the address of the first node
  - The previous field of the first node store the address of the last node


300
10
200
100
20
300
200
30
100
100
200
300

- Inserting a new node in circular doubly linked list
  - Node inserting at the beginning
  - Node inserting at the end

- **Routine for insertion at the beginning**

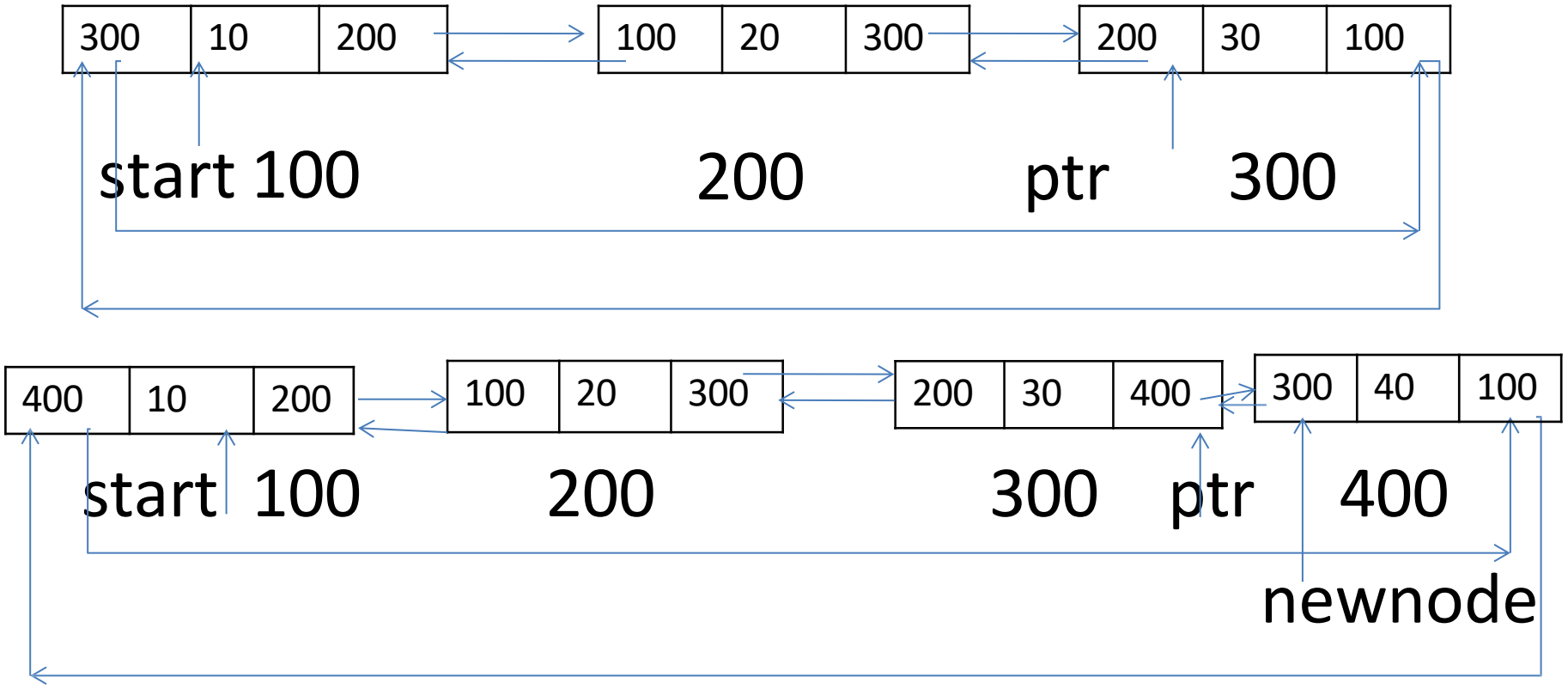




Routine for insertion at the beginning

Step:1 if avail== null
write overflow
go to step 13
Step:2 set newnode=avail
Step:3 set avail=avail→next
Step:4 set newnode→data=val
Step:5 set ptr=start
Step:6repeat step 7 while(ptr→next !=start)
Step:7 ptr=ptr→next
end of loop
Step:8 set ptr→next=newnode
Step:9 set newnode→prev=ptr
Step:10 set newnode→next=start
Step:11 set start→prev=newnode
Step:12 set start=newnode
Step:13 exit

Routine for insertion at the end

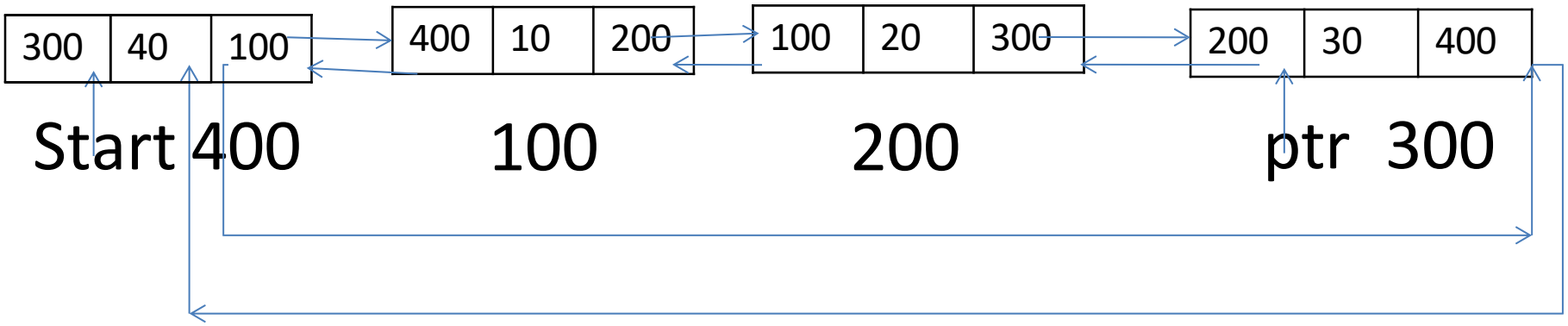


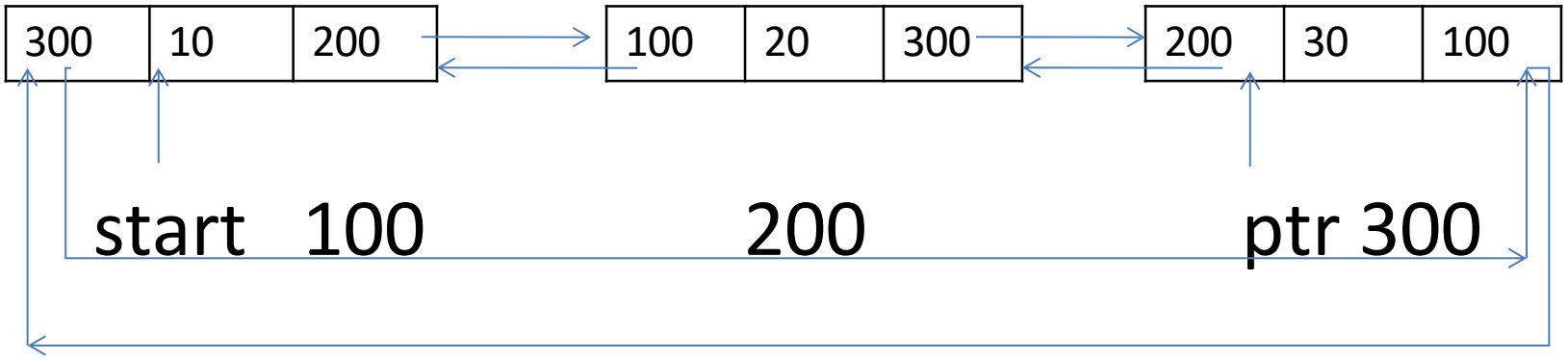
Routine for insertion at the end

Step:1 if avail== null
write overflow
go to step 12
Step:2 set newnode=avail
Step:3 set avail=avail→next
Step:4 set newnode→data=val
Step:5 set ptr=start
Step:6repeat step 7 while(ptr→next !=start)
Step:7 ptr=ptr→next
end of loop
Step:8 set ptr→next=newnode
Step:9 set newnode→prev=ptr
Step:10 set newnode→next=start
Step:11 set start→prev=newnode
Step:12 exit

- Deleting a new node in circular doubly linked list
  - Node deletion at the beginning
  - Node deletion at the end

- **deleting at the beginning**





Step:1 if start= null

write underflow

goto step 9

endif

Step:2 set ptr=start

Step:3 repeat step 4 while(ptr→next!=start)

Step:4 set ptr=ptr→next

end loop

Step:5 set ptr→next=start→next

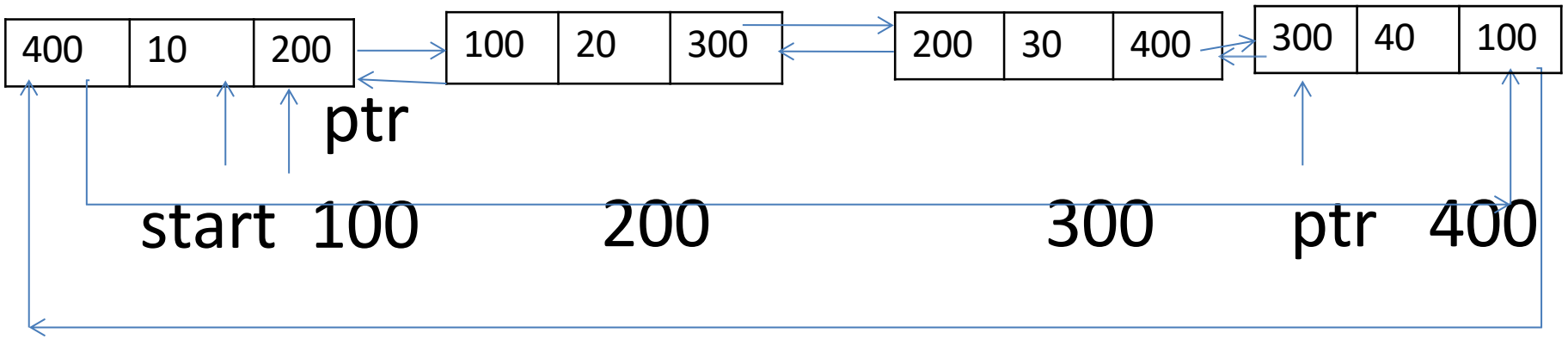
Step:6 set start→next→prev=ptr

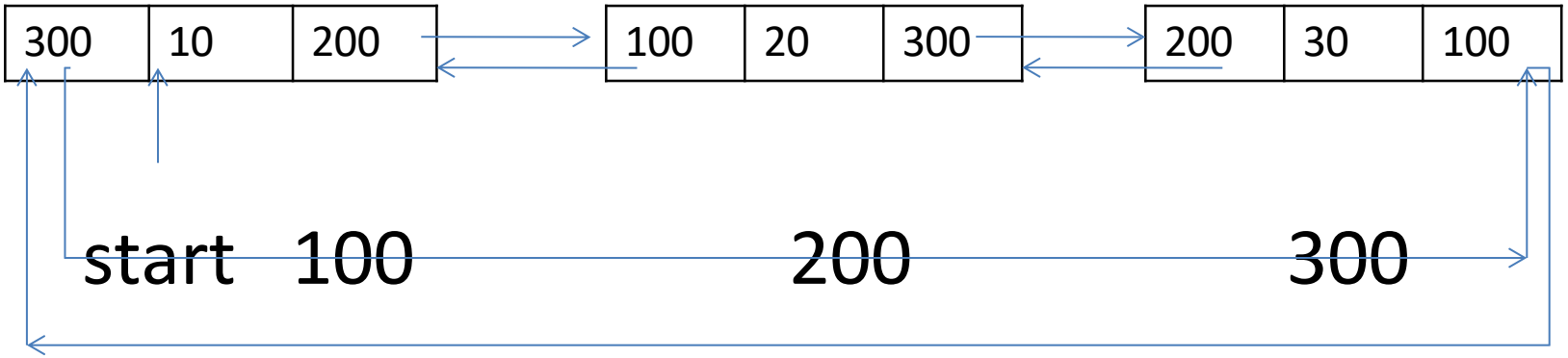
Step:7 free start

Step:8 set start=ptr→next

Step:9 exit

deletion at the end





Step:1 if start= null

write underflow

goto step 8

endif

Step:2 set ptr=start

Step:3 repeat step 4 while(ptr→next!=start)

Step:4 set ptr=ptr→next

end loop

Step:5 set ptr→prev→next=start

Step:6 set start→prev=ptr→prev

Step:7 free ptr

Step:8 exit

Application of linked list

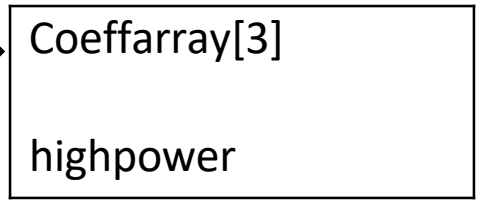
- Polynomial manipulation
- Radix sort
- multilist

Polynomial manipulation

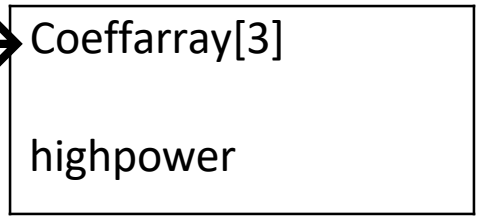
Poly1= $3x^2+2x+1$

Poly2=$2x^2+3x+2$

Poly1→



Poly2→



Polynomial manipulation

Declaration

Typedef struct

{

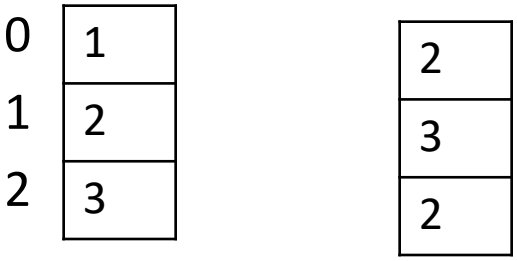
Int coeffarray[maxdegree+1];

Int highpower;

}polynomial;

Poly1 = $3x^2+2x+1$ $2x^3+2x+1$

Poly2 = $2x^2+3x+2$ $2x^2+3x+4$

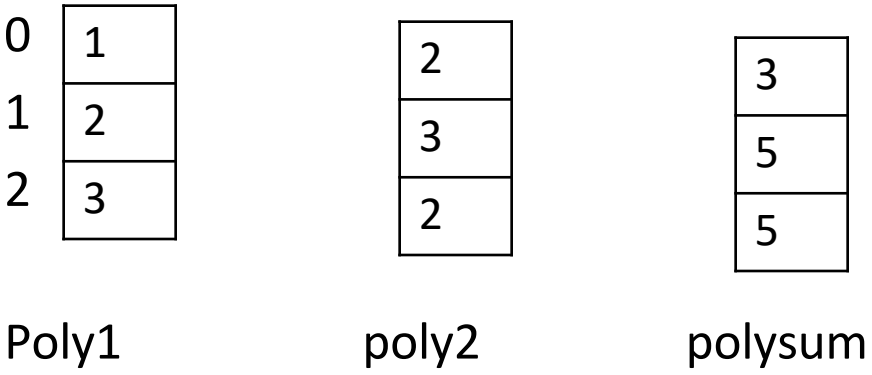


Poly1 poly2

Poly1 = $3x^2+2x+1$ $2x^3+2x+1$

Poly2 = $2x^2+3x+2$ $2x^2+3x+4$

Polysum = $5x^2+5x+3$ $2x^3+2x^2+5x+5$



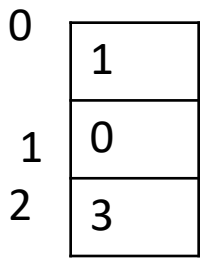
```
Void zeropolynomial(polynomial poly)
{
Int i;
For(i=0;i<=maxdegree;i++)
    Poly→coeffarray[i]=0;
Poly→highpower=0;
}
```

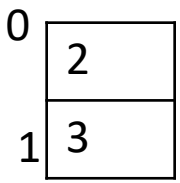
Polynomial addition

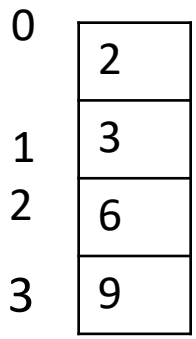
```
Void addpolynomial(const polynomial poly1,const polynomial
    poly2,polynomial polysum)
{
Int i;
Zeropolynomial(polysum);
Polysum→highpower=max(poly1→highpower,poly2→highpower);
For(i=polysum→highpower;i>=0;i--)
    polysum→coeffarray[i]=poly1→coeffarray[i]+poly2→coeffarray[i];
}
```

Poly1 $= 3x^2+1$

Poly2 $=3x+2$







Poly1 = $3x^2+1$

Poly2 =$3x+2$

Polyprod =$6x^2+2+9x^3+3x$

= $9x^3+6x^2+3x+2$

$3x^2+1$

[0] i=0,j=0→2

[1]i=0,j=1→3

[1] i=1,j=0→3+0=3

[2]i=1,j=1→0

[2]i=2,j=0 →0+6

[3] i=2,j=1→9

0	1
1	0
2	3

0	2
1	3

0	2
1	3
2	6
3	9

```
Void multpolynomial(const polynomial poly1,const polynomial
    poly2,polynomial polyprod)
{
int i,j;
Zeropolynomial(polyprod);
Polyprod→highpower=poly1→highpower+poly2→highpower;
If(polyprod→highpower>maxdegree)
    error("exceed array size);
else
For(i=0;i<=poly1→highpower;i++)
For(j=0;j<=poly2→highpower;j++)
    polyprod→coeffarray[i+j]+=poly1→coeffarray[i]*poly2→coeffarray[j];
}
```

```
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>
#define NULL 0
typedef struct list
{    int no;
struct list *next;
}LIST;
LIST *p,*t,*h, *y,*ptr,*pt;
void create( void );
void insert( void );
void delet( void );
void display ( void );
int j,pos,k=1,count;
void main()
{          int n,i=1,opt;
clrscr();
p = NULL;
printf( "Enter the no of
nodes :\n " );
scanf( "%d",&n );
count = n;
while( i <= n)
{          create();
i++;
}
```

```
printf("\nEnter your
    option:\n");
printf("1.Insert \t 2.Delete \t
    3.Display \t 4.Exit\n");
do
{
scanf("%d",&opt);
switch( opt )
{
case 1:
insert();
count++;
break;
case 2:
delet();
count--;
if ( count == 0 )
{
printf("\n List is empty\n");
}
break;
case 3:
printf("List elements are:\n");
display();
break;
}
printf("\nEnter your option \n");
}while( opt != 4 );
getch();
}
void create ( )
{if( p== NULL )
{
p = ( LIST * ) malloc ( sizeof ( LIST )
);
```

```
printf( "Enter the element:\n"
    );
scanf( "%d",&p->no );
p->next = NULL;
h = p;
}
else
{   t= ( LIST * ) malloc (sizeof(
    LIST ));
printf( "\nEnter the element");
scanf( "%d",&t->no );
t->next = NULL;
p->next = t;
p = t;
}
}
```

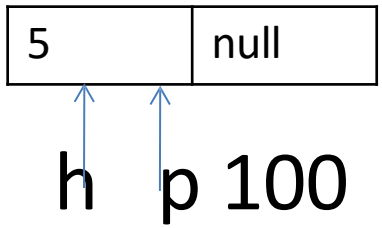
```
void insert()
{           t=h;
p = ( LIST * ) malloc ( sizeof(LIST) );
printf("Enter the element to be
insrted:\n");
scanf("%d",&p->no);
printf("Enter the position to insert:\n");
scanf( "%d",&pos );
if( pos == 1 )
{
h = p;
h->next = t;
}
else
{
for(j=1;j<(pos-1);j++)
t = t->next;
p->next = t->next;
t->next = p;
t=p;
}
}
```

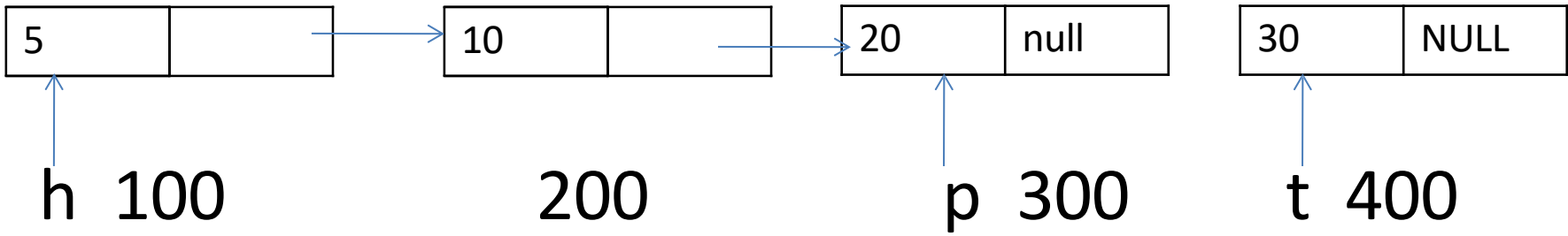
void delet()

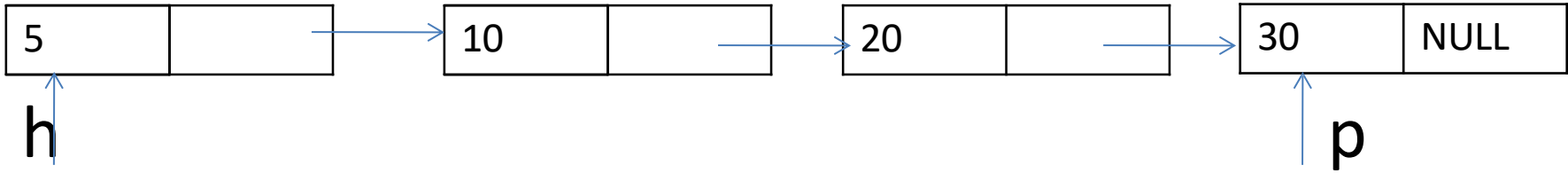
```
printf("Enter the position to
    delete:\n");
scanf( "%d",&pos );
if( pos == 1 )
{
h = h->next ;
}
else
{    t= h;
for(j=1;j<(pos-1);j++)
t = t->next;
pt=t->next->next;
free(t->next);
t->next= pt;
}
}
```

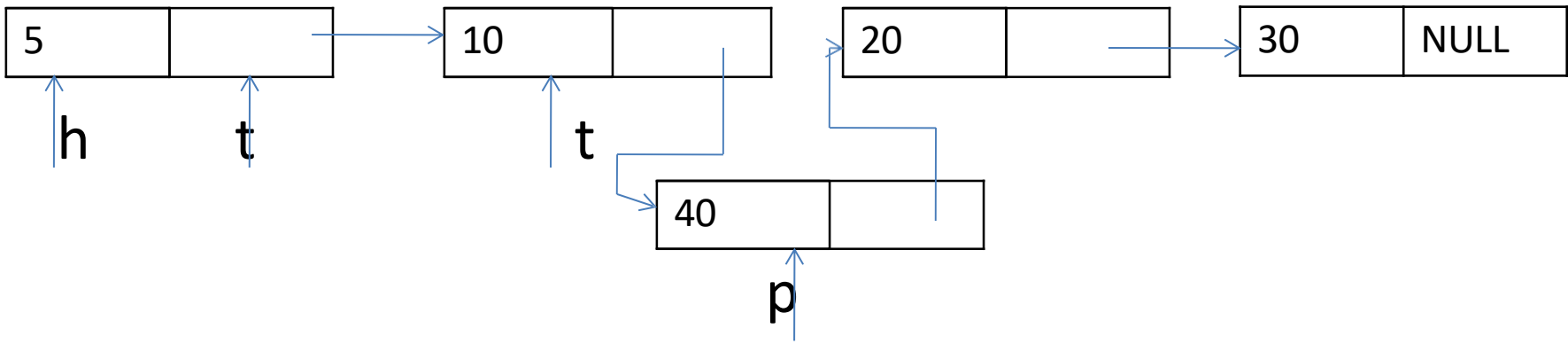
void display()

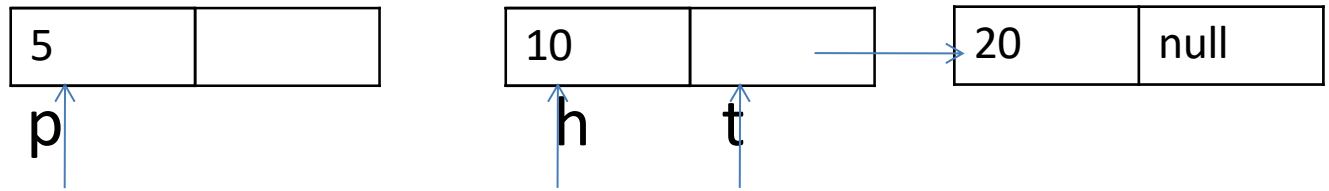
```
{            t = h;
while( t->next != NULL )
{
printf("\t%d",t->no);
t = t->next;
}
printf( "\t %d\t",t->no );
}
```

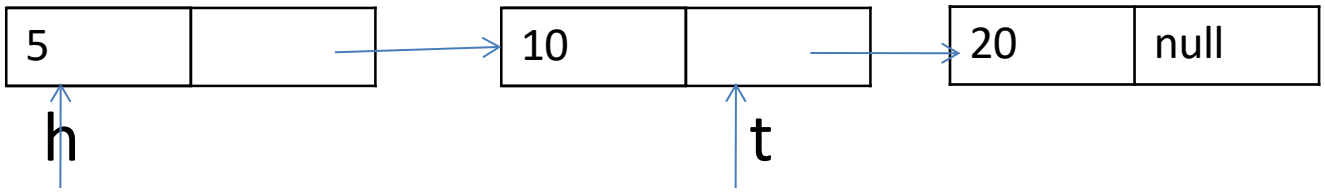

5
null
h
p 100

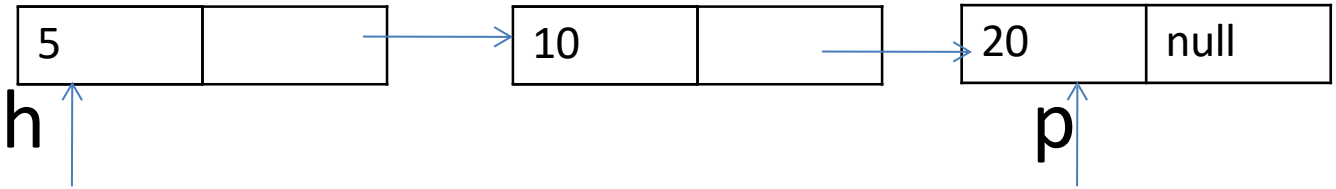

5
10
20
null
30
NULL
h 100
200
p 300
t 400

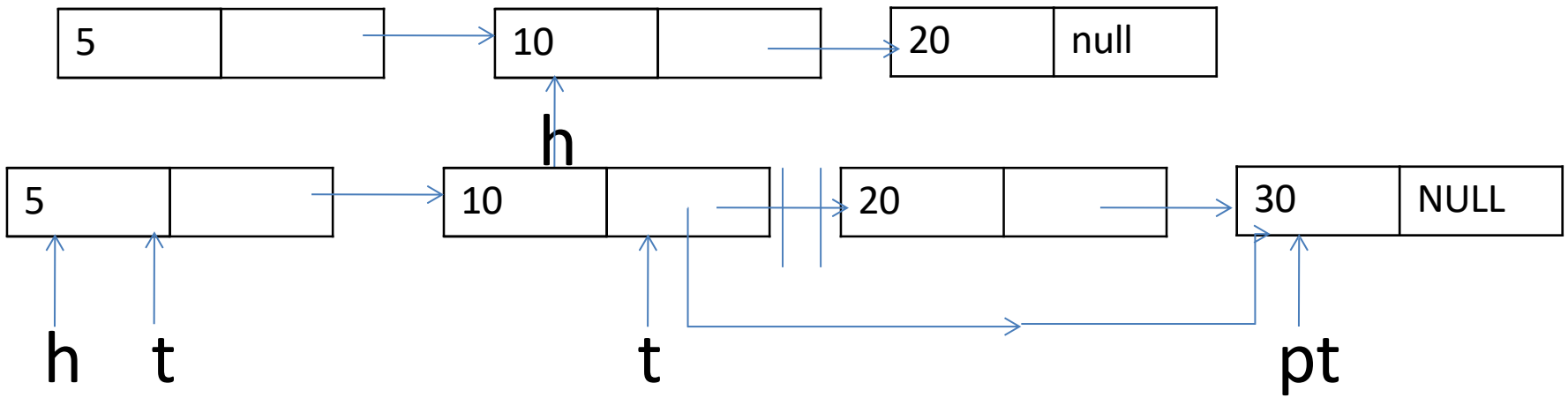

5
10
20
30
NULL
h
p


5
10
20
30
NULL
h
t
t
40
p


5
10
20
null
p
h
t


5
10
20
null
h
t


5
10
20
null
h
p


5
10
20
null
h
5
10
20
30
NULL
h
t
t
pt